Design and Implementation of a Web-Based Registration System for Northport University

CS 5910 - Design & Implementation

Fall 2025

Prof. Gupta

Team Members

Michael Covelli - Team Lead

email: mcovelli@oldwestbury.edu Phone: 516-695-4162

Jackson Beach

email: jacksonbeach39@gmail.com Phone: 631-464-7191

Michael Bekoe

email:mdarko2@oldwestbury.edu Phone:631-514-1037

Weekly Log

Team Member	Time	Date	Place	Present/ Absent	What was discussed	What was decided
Mike C.	4:20pm	9/2/25	NAB 0109	Present	Division of duties	Faculty Requirements and constraints
Jackson B.				Present		Student Requirements and constraints
Mike B.				Present		Admin Requirements and constraints
Mike C.	3:00pm	9/4/25	NAB 0109	Present	Corrections to SRS. Add project summary, additions to requirements and constraints	Write project summary, Add to Faculty requirements and constraints
Jackson B.				Present		Add to Student requirements and constraints
Mike B.				Present		Add to Admin requirements and constraints
Mike C.	4:00pm	9/9/25	NAB 0109	Present	• Corrections to Use cases. • Additions to use cases • Reformat use case tables • Consolidate multi-user use cases	Reformate use case tables Add to faculty use cases Consolidate multi-user use cases
Jackson B.				Present		Add to Student use cases
Mike B.				Present		Add to Admin

						use cases
Mike C.	4:00pm	9/11/25	NAB 0109	Present	• Clarified needed use cases • Start list of entities/ relationships	Add to faculty use cases Start list of entities and relationships
Jackson B.				Present		Add to Student use cases
Mike B.				Present		Add to Admin use cases
Mike C.	5:00pm	9/14/25	ZOOM	Present	• Reviewed use cases • Discussed entity/ relationships	
Jackson B.				Present		
Mike B.				Present		
Mike C.	2:40pm	9/18/25	NAB 0109	Present	• Discussed entity/ relationships	Organized and Added to EERD
Jackson B.				Present		Added to EERD
Mike B.				Present		
Mike C.	2:40pm	9/23/25	NAB 0109	Present	• Met with the professor	Made corrections to Entities
Jackson B.				Present		Made corrections to Entities
Mike B.				Present		Made corrections to Entities
Mike C.	2:30pm	10/7/25	Zoom	Present	Discussed php and html codes	Attempted to get everyone on EC2 and to link VS Code and phpmyadmin to

						EC2
Jackson B.				Present		Worked on Wireframes
Mike B.				Present		Worked on Wireframes
Mike C.	2:30pm	10/13/25	Zoom	Present	Discussed php and html codes for dashboards and homepage	Worked on database
Jackson B.				Present		Implemented homepage
Mike B.				Present		Sample of student dashboard
Mike C.	2:30pm	10/20/25	Zoom	Present	Further discussed php and html codes for dashboards	Worked on database, changes to admin dashboard php and html, fixed reset_password.php, finalized Word document with business rules, relational schema, create tables queries
Jackson B.				Present		Continue work on Student dashboard
Mike B.				Present		Continue work on Course Section view html

Table of Contents

1. Project Summary 7
2. System Required Specifications 8
    2.1. Student (Explicitly Required) 8
        2.1.1. Login and Profile 8
        2.1.2. View (read only access) 8
        2.1.3. Actions 9
        2.1.4. Explicitly Forbidden (constraints) 10
    2.2. Faculty (Explicitly Required) 11
        2.2.1. Login and Profile 11
        2.2.2. View (All Student info; read only access; faculty cannot modify this data) 11
        2.2.3. Actions 12
        2.2.4. Explicitly Forbidden (constraints) 13
    2.3. Administration (Explicitly Required) 13
        2.3.1. Login and Profile 13
        2.3.2. View(Read only access) 14
        2.3.3. Actions 15
        2.3.4. Explicitly Forbidden (constraints) 17
3. Business Rules 20
    3.1. Student Business Rules 20
        3.1.1. Login & Profile Rules 20
        3.1.2. Viewing Rules (Read-Only Access) 20
        3.1.3. Action Rules (Allowed Operations) 21
        3.1.4. Explicitly Forbidden (Constraints) 21
    3.2. Faculty Business Rules 22
    3.2.1. Login & Profile Rules 22
    3.2.2. Viewing Rules (Read-Only Access) 22
    3.2.3. Action Rules (Allowed Operations) 23
    3.2.4. Explicitly Forbidden (Constraints) 23
    3.3. Administration Business Rules 24
    3.3.1. Login & Profile Rules 24
    3.3.2. Viewing Rules (Read-Only Access) 24
    3.3.3. Action Rules (Allowed Operations) 24
    3.3.4. Explicitly Forbidden (Constraints) 25
4. Use Cases 26
    4.1. Student Use Cases 26

4.1.1. Unsuccessfully add a course-section 26
4.1.2. Successfully add a course-section 28
4.1.3. Successfully drop a course-section 29
4.1.4. Unsuccessfully drop a course-section 31
4.1.5. Successfully Declare Major 32
4.1.6. Successfully Declare Minor 32
4.1.7. Unsuccessfully Declare Major 33
4.1.8. Unsuccessfully Declare Minor 34
4.2. Faculty Use Cases 35
4.2.1. View Current Semester Schedule 35
4.2.2. View Next Semester Schedule 36
4.2.3. Successfully Assign Grades 37
4.2.4. Unsuccessfully Assign Grades 38
4.2.5. Successfully Record Attendance 40
4.2.6. Unsuccessfully Record Attendance 41
4.2.7. 42
Successfully Send Email Notifications to Students 43
4.2.8. Unsuccessfully Send Email Notifications to Students 44
4.3. Administration Use Cases 46
4.3.1. Manage User Accounts(Student & Faculty) 46
4.3.2. Course Management (Create | Update | Delete | Courses) 47
4.3.3. Enrollments section 49
4.3.4. System security & Access Control 50
4.3.5. Successfully Cancel course Section 51
4.3.6. Unsuccessfully Cancel Course Section 52
4.3.7. Successful Pre-Semester Update(Room/Faculty Change) 53
4.3.8. Unsuccessful Pre-Semester updates (Room/Faculty Change) 54
4.3.9. Successfully create Course 55
4.3.10. Unsuccessfully Create Course 56
Unsuccessfully Create Course 56
4.3.11. Successfully Update Course 57
4.3.12. Unsuccessfully Update Course 58
4.3.13. Successfully Update a student's Grade in Course-Section 59
4.3.14. Unsuccessfully update a student's Grade in Course-Section 60
4.3.15. Successfully Assign Time Slots & Rooms 61
4.3.16. Unsuccessfully Assign Time Slots & Rooms 62
4.3.17. Successfully create Master Schedule Entry 62
4.3.18. Successfully Update Master Schedule Entry 63
4.3.19. Unsuccessfully create Master Schedule Entry 64

4.3.20. Unsuccessfully Update Master Schedule Entry 64
4.3.21. Successfully Create Department Major/Minor 65
4.3.22. Successfully Update Department Major/Minor 66
4.3.23. Unsuccessfully Create Department Major/Minor 68
4.3.24. Unsuccessfully Update Department Major/Minor 69
4.3.25. Successfully Create Department 71
4.3.26. Unsuccessfully Create Department 71
4.3.27. View All Users 72
4.3.28. Create Course Prerequisite 73
4.3.29. Update Course Prerequisite 74
4.4. Multi-User Use Cases 76
4.4.1. View Master Schedule 76
4.4.2. View Degree Progress 76
4.4.3. View Advisee List 78
4.4.4. Attempts to change student information 79
4.4.5. Successfully View Course Prerequisites 80
4.4.6. Unsuccessfully View Course Prerequisites 82
4.4.7. View Class Roster 83
4.4.8. Attempt to Change Schedule (Forbidden) 84
4.4.9. Unsuccessful Login/Password Reset 85
4.4.10. Successful Login 86
4.4.11. View Student Info 88
4.4.12. View Semester Grades 91
4.4.13. View GPA and Unofficial Transcript 92
4.4.14. View Degree Audit 93
4.4.15. View holds 94
4.4.16. Successfully Reset Password 96
4.4.17. Change Password 98
4.4.18. View Department 99
4.4.19. View Courses of Department 100
4.4.20. View Majors/Minors of Department 101
4.4.21. View Majors/Minors Course Requirements 102
4.4.22. View Schedule for Fall 2025 103
4.4.23. View Schedule for Spring 2026 105
4.4.24. Successfully Search for Advisor's Info 107
4.4.25. Unsuccessfully Search for Advisor's Info 108
4. Entity/Relationships 109
5. EERD 120
6. Schemas 122

7. Create Table Queries 126

1. Project Summary

The Systems Design Project for CS 5910 – Design & Implementation (Fall 2025) focuses on the design and implementation of a web-based registration system for Northport University. The system is intended to modernize how students, faculty, and administrators interact with course registration, scheduling, and academic recordkeeping. Students will be able to log in securely, declare majors and minors, register for courses, view their schedules, grades, GPA, and degree progress, and access course prerequisites. Faculty will use the system to log in, manage their own course rosters, assign grades within approved windows, track attendance, and advise students, while Administration will oversee master schedule creation, course-section assignments, department management, and enforcement of institutional rules.

The project scope includes the creation of an extended Entity Relationship Diagram (ERD), the development of a database schema, and the population of the system with both historical and upcoming semester data (Fall 2025, Spring 2026, and 4–5 prior years). Business rules are central to the system's design: students cannot exceed credit caps (16 credits for full-time, 8 for part-time), cannot retake passed courses, and cannot bypass prerequisites or account holds; faculty are limited in teaching load and department affiliations; and administrators must enforce seat capacities, scheduling conflicts, and the requirement that no course-section may run with fewer than five enrolled students. To ensure integrity, all user IDs will be generated as randomized 5–6 digit identifiers that are immutable, and all system changes (schedule updates, grade modifications, cancellations) will be logged with full traceability.

This system is being developed by a three-member team consisting of Michael Covelli (Team Lead), Jackson Beach, and Michael Bekoe — working collaboratively to meet strict design and reporting standards. The team leader is responsible for setting goals with the client, ensuring project milestones are met, and compiling weekly typed reports in a formal binder. Each member contributes according to their strengths: requirements analysis, database design, and implementation. The final deliverables will include the extended ERD, a working database and populated schema, the functional registration system, and a complete Software Requirements Specification (SRS) with use cases, constraints, and documented logs of decisions and progress.

2. System Required Specifications

2.1. Student (Explicitly Required)

2.1.1. Login and Profile

S-1. Students can login to the system with a unique Username and Password.

S-2. Students can have a status of full-time or part-time, determined by a threshold number of courses.

S-3. Students can only belong to 1- 2 major(s) and one minor of their choice, which can only be changeable up to their final year.

S-4. Students can be either Graduate Students or Undergraduate students.

S-5. Graduate students can be MS or PhD candidates.

S-6. Undergraduate students can be Associates or Bachelor's degree candidates.

S-7. Undergraduate students can be either Freshman, Sophomore, Junior or Senior.

2.1.2. View (read only access)

S-8. Students can view major course/minor course prerequisites.

S-9. Students can view their current and past semester schedules.

S-10. Students can view all available courses.

S-11. Students can view a list of majors and minors.

S-12. Students can view major/minor requirements

S-13. Students can view all the grades they have gotten from each course they have taken.

S-14. Students can view their semester GPA

S-15. Students can view their cumulative GPA.

S-16. Students can view degree progress for major and minor.

S-17. Students can view account holds.

S-18. Students can view Master Schedule

- S-a. Faculty Name
- S-b. building/room no.
- S-c. Time slot (dates and times)
- S-d. Available Seats.
- S-e. Course Name.
- S-f. Department
- S-g. Major/Minor
- S-h. CRN

2.1.3. Actions

S-19. Students can login into system

S-20. Students can declare a major.

S-21. Students can search Master Schedule

- S-a. Department
- S-b. Major/Minor
- S-c. Course Name
- S-d. Faculty Name
- S-e. Time slot (date/time)
- S-f. Building/room no.
- S-g. Available seats.

S-h. CRN

S-22. Students can search and browse for courses designated by their major and minor, as well as others.

S-23. Students can add a course-section to an upcoming semester.

S-24. Students can drop a course-section.

S-25. Students can check assignment and exam grades and notifications from their professors.

S-26. Students can change their password if they have forgotten it or if they are required to change it at any point.

S-27. Students can change their major and/or minor.

2.1.4. Explicitly Forbidden (constraints)

S-C1. Students cannot change their username.

S-C2. Students cannot register for a course section if there is a hold (restriction to access) on their account

S-C3. Students cannot register for a course if they have not met course's prerequisites.

S-C4. Students cannot take more than 16 credits (4 courses) in a given semester if they are full-time.

S-C5. Students cannot take more than 8 credits (2 courses) in a given semester if they are part-time.

S-C6. Students cannot retake any courses that they have already passed if they receive a C or higher in said courses.

S-C7. Students cannot add or drop a course section outside a time window in any semester.

S-C8. Students cannot add a course to their schedule if no available seats (course has reached max no. of students).

S-C9. Students cannot take 2 course sections in the same time slot (days/periods) if scheduling conflict occurs.

S-C10. Students cannot enroll in graduate level courses as an undergraduate.

S-C11. Students cannot enroll in undergraduate level courses as a graduate student.

S-C12. Students cannot change any grade they have received for a course.

S-C13. Students cannot update attendance.

S-C14. Students cannot change any user IDs (which are fixed).

S-C15. Students cannot see any other user's info (private).

S-C16. Students cannot view Faculty and Administration specific information

S-Ca. Faculty schedule

S-Cb. Faculty list of advisees

S-Cc. course-section rosters

2.2. Faculty (Explicitly Required)

2.2.1. Login and Profile

F-1. Faculty can login to the system with a unique Username and Password.

F-2. Faculty can have a status of Full Time or Part Time.

F-3. Faculty can belong to one to three departments.

2.2.2. View (All Student info; read only access; faculty cannot modify this data)

F-4. Faculty can view their present semester schedule.

F-5. Faculty can view their next semester schedule.

F-6. Faculty can view their course-section roster(s) and their information.

F-a. Students current semester schedule

F-b. Students next semester schedule

F-c. Students account holds (Health, Financial, Academic)

F-d. Students Major/Minor

F-e. Students department

F-f. Email

F-g. Grades

F-7. Faculty can view their list of their advisees and advisee information.

F-8. Faculty can view Master Schedule

F-a. Faculty

F-b. Building/room no.

F-c. Time slot (date/time)

F-d. CRN

F-e. Course Name

F-9. Faculty can view a list of majors and minors

F-10. Faculty can view course section information (faculty, building/room no., dates and times)

F-11. Faculty can view major and minor requirements.

F-12. Faculty can view course prerequisites.

F-13. Faculty can view a list of departments.

2.2.3. Actions

F-14. Assign grades to their own course-section(s) within one week after final exam.

F-15. Record attendance on the days/periods of their own course-section(s).

F-16. Send email notifications to students in their own course-section roster(s).

2.2.4. Explicitly Forbidden (constraints)

F-C1. If Full time, Faculty cannot teach more than 2 course-sections.

F-C2. Faculty cannot belong to more than 3 department(s).

F-C3. Faculty cannot teach outside your own department(s).

F-C4. Faculty cannot assign grades to course-sections they are not teaching.

F-C5. Faculty cannot record attendance for course-sections they are not teaching.

F-C6. Faculty cannot teach 2 or more courses in the same time slot.

F-C7. Faculty cannot have more than 15 advisees. (depending on number of faculty/students)

F-C8. Faculty cannot change User ID or login credentials.

F-C9. Faculty cannot change their schedule.

F-C10. Faculty cannot modify student personal information, login credentials, schedules, grades, transcripts or holds.

F-C11. Faculty cannot change student schedules.

F-C12. Faculty cannot change student grades without approval.

2.3. Administration (Explicitly Required)

2.3.1. Login and Profile

A-1. Administration can log into the system with Username(email) and password.

A-2. Administration can have a Status of Full-time only.

A-3. Administration has elevated privileges and security Level to:

- A-a. View Faculty and Student information.
  - A-i. Name
  - A-ii. Department
  - A-iii. Major/Minor
  - A-iv. Holds
  - A-v. Unofficial transcript
  - A-vi. Semester Grades
  - A-vii. Semester Schedule
- A-b. Update Faculty and Student information.

2.3.2. View(Read only access)

A-4. Administration can view student information, Unofficial transcript, Holds on accounts, Past semester schedule, current semester schedule, next semester schedule.

A-5. Administration can view Major and minor course requirements

A-6. Administration can view students' semester grades after (mid-term exams or final exams) have been submitted to the portal.

A-7. Administration can view all list of Departments:

- A-a. Majors Departments
- A-b. Minors Departments.

A-8. Administration can view all course pre-requisites.

A-9. Administration can view faculty current semester schedules, next semester schedules, and course-section rosters.

A-10. Administration can view the Faculty list of advisees and their information.

A-11. Administration can view all users.

2.3.3. Actions

A-12. Administration can create/update Users(which includes Students, Faculty, and Admin) based on these factors:

- A-a. UserID
- A-b. Full Name
- A-c. Address
- A-d. Phone
- A-e. Date of Birth(DOB)
- A-f. Part/Full Time for:
  - A-i. Undergraduate(Freshman,Sophomore,Junior,Senior)
  - A-ii. Graduate(MS/PhD)

A-g. Email

A-h. Password.

A-13. Administration can update user Information.

A-a. Contact info

A-b. Schedules

A-c.

A-14. Administration can create a Master Schedule for every Department for a given semester (Fall 2025/Spring 2026) based on these factors:

A-a. CRN(Unique course ID), Course name, Section, Faculty, Time slot, Building (room number), Available seats.

A-15. Administration can update the Master schedule per semester.

A-16. Administration can create new courses or departments (with limited privileges)

A-17. Administration can create new course-section

A-18. Administration must ensure there are no time conflicts for both Faculty and classrooms on the master schedule:

A-a. To ensure efficient resource allocation, each faculty is limited to instructing one course-section concurrently, and each classroom can only accommodate a single course-section at any given time.

A-19. Administration can assign faculty to course-sections

A-20. Administration can assign a Building/Room no. to a course-sections.

A-21. Administration can assign available seats to a course-section

A-22. Administration can assign time slots to course-section

A-23. Administration can create new prerequisites or course

A-24. Administration can create new major and minor department course requirements.

A-25. Administrative Cancellations**:**

A-a. Course sections can be canceled within a time window, requiring student notification if registered.

A-26. Pre-Semester Updates**:**

A-a. Building/room numbers for course sections must be updated before the semester.

A-b. Faculty assignments for course sections must be updated before the semester.

A-27. Administration can add/remove students from course sections.

A-28. Administration can update grades for completed course sections (with limited privileges).

A-29. The administration must drop a course section at least one week prior to the start of the semester if enrollment is less than five.

A-30. All course changes must be logged.

A-a. Who made the change.

A-b. What was changed.

A-c. When it was changed.

2.3.4. Explicitly Forbidden (constraints)

A-C1. The administration cannot assign full-time faculty to more than two course sections.

A-C2. The administration cannot assign part-time faculty to more than one course section.

A-C3. The administration cannot schedule two course sections in a given room, thereby avoiding room conflicts.

A-C4. The administration cannot alter user information without a user request.

A-C5. The administration cannot drop a course section after the semester has started.

A-C6. The administration cannot assign faculty to a course-section outside of their department.

A-C7. Administration cannot override the 5-person course-section minimum rule.

A-C8. Administration cannot override seat limits

A-C9. Administration cannot override credit limits

A-Ca. No more than 16 credits for full time students.

A-Cb. No more than 8 credits for part-time students

A-C10. Administration cannot assign more than one course-section to the same CRN.

A-C11. Administration cannot view or modify passwords.

A-C12. Administration cannot create duplicate User IDs.

A-C13. Administration cannot change User IDs.

A-C14. Administration cannot delete users once created (only deactivate or archive).

A-C15. Administration cannot assign more than the maximum number of seats for a course-section.

A-C16. Administration cannot change attendance records for students.

A-C17. Administration cannot assign faculty to more than one course-section in the same time slot.

A-C18. Administration with View-Only security cannot update any data.

A-C19. Administration cannot bypass any course-section prerequisite requirements.

A-C20. Administration cannot assign a student to a course-section without fulfilling course-section prerequisite requirements.

A-C21. Administration cannot assign a student to a course-section if a student has previously passed the course-section.

A-C22. Administration cannot change grade information without

authorization. (e.g. grade change name)

A-C23. Administration cannot update faculty roster, advisees or schedules without approval.

3. Business Rules

3.1. Student Business Rules

3.1.1. Login & Profile Rules

1. Each student must log in with a unique username (email) and password.
2. Each student's username and user ID are permanent and cannot be changed.
3. A student's status (full-time or part-time) is determined by their credit or course load threshold.
4. A student must belong to at least one and no more than two majors, and at most one minor.
5. A student may change major/minor only until their final academic year.
6. Each student must be classified as Undergraduate or Graduate.
7. Undergraduate students must have a degree type of *Associate* or *Bachelor*.
8. Graduate students must have a program type of *MS* or *PhD*.
9. Undergraduate students must have a year classification: *Freshman, Sophomore, Junior, or Senior*.

3.1.2. Viewing Rules (Read-Only Access)

10. Students can view major/minor requirements and course prerequisites.
11. Students can view their current, next, and past semester schedules.
12. Students can view all available courses across departments.
13. Students can view all majors, minors, and departments.
14. Students can view their course grades (per course, per semester).
15. Students can view their cumulative GPA and semester GPA.
16. Students can view their degree progress (credits completed and remaining).
17. Students can view account holds (academic, financial, or health).
18. Students can view the Master Schedule, which includes:
    - Faculty Name
    - Building and Room Number
    - Time Slot (dates/times)
    - Available Seats
    - Course Name

- Department / Major / Minor
- CRN

3.1.3. Action Rules (Allowed Operations)

19. Students can log in/out of the system.
20. Students can declare or change their major/minor (within allowed timeframe).
21. Students can search the Master Schedule by:
    - Department
    - Major/Minor
    - Course Name
    - Faculty Name
    - Time Slot
    - Building/Room No.
    - Available Seats
    - CRN
22. Students can add course sections to their upcoming semester schedule.
23. Students can drop course sections within the designated time window.
24. Students can check assignment and exam grades and receive faculty notifications.
25. Students can change their password, either voluntarily or if required by the system.

3.1.4. Explicitly Forbidden (Constraints)

26. Students cannot change usernames or user IDs.
27. Students cannot register for courses if they have any active hold on their account.
28. Students cannot register for a course without meeting all prerequisites.
29. Full-time students cannot exceed 16 credits (4 courses) per semester.
30. Part-time students cannot exceed 8 credits (2 courses) per semester.
31. Students cannot retake a course if they received a grade of C or higher.
32. Students cannot add/drop courses outside the semester's allowed time window.
33. Students cannot register for a course section with no available seats.
34. Students cannot enroll in two courses that overlap in time slots.
35. Undergraduate students cannot enroll in graduate-level courses.
36. Graduate students cannot enroll in undergraduate-level courses.
37. Students cannot modify or change any grades they have received.

38. Students cannot update attendance records.
39. Students cannot access or view other users' information.
40. Students cannot view faculty or administrative data, including:
    - Faculty schedules
    - Faculty advisee lists
    - Course-section rosters

3.2. Faculty Business Rules

3.2.1. Login & Profile Rules

41. Each faculty member must log in with a unique username and password.
42. Faculty may be Full-Time or Part-Time.
43. Faculty may belong to one to three departments only.

3.2.2. Viewing Rules (Read-Only Access)

44. Faculty can view their current semester schedule.
45. Faculty can view their next semester schedule.
46. Faculty can view their course-section rosters (students and details).
47. Faculty can view advisees and advisee information.
48. Faculty can view students' schedules (current/next semester), holds, major/minor, department, email, and grades.
49. Faculty can view the Master Schedule with full course, faculty, building, and time details.
50. Faculty can view all departments, majors, minors, and their requirements.
51. Faculty can view course prerequisites.

3.2.3. Action Rules (Allowed Operations)

52. Faculty can assign grades for their own course sections within one week after final exams.
53. Faculty can record attendance for their own course sections on scheduled days/periods.
54. Faculty can send email notifications to students in their own course-section rosters.

3.2.4. Explicitly Forbidden (Constraints)

55. Full-time faculty cannot teach more than two course sections per semester.
56. Part-time faculty cannot teach more than one course section per semester.
57. Faculty cannot belong to more than three departments.
58. Faculty cannot teach outside their department(s).
59. Faculty cannot assign grades to course sections they do not teach.
60. Faculty cannot record attendance for course sections they do not teach.
61. Faculty cannot teach two or more courses in the same time slot.
62. Faculty cannot have more than 15 advisees.
63. Faculty cannot change user IDs or login credentials.
64. Faculty cannot modify their schedules once assigned.
65. Faculty cannot alter student personal data, grades, schedules, or holds.
66. Faculty cannot change student grades without approval.

3.3. Administration Business Rules

3.3.1. Login & Profile Rules

67. Administration must log in using username (email) and password.
68. Administrators must have Full-Time status.
69. Administrators have a defined security level (VIEW or UPDATE) determining access privileges.

3.3.2. Viewing Rules (Read-Only Access)

70. Administrators can view all faculty and student information, including:
    - Name, Department, Major/Minor, Holds, Transcript, Semester Grades, and Schedule.
71. Administrators can view major/minor requirements and course prerequisites.
72. Administrators can view all department lists (for majors and minors).
73. Administrators can view faculty schedules, course rosters, and advisee lists.
74. Administrators can view users, their roles, and statuses.

3.3.3. Action Rules (Allowed Operations)

75. Administrators can create and update users (Student, Faculty, Admin) with full demographic and status data.
76. Administrators can update contact info and schedules.
77. Administrators can create Master Schedules for each semester and department, including:
    - CRN, Course Name, Section, Faculty, Time Slot, Room, Seats.
78. Administrators can update Master Schedules and course sections each semester.
79. Administrators must ensure no room or faculty time conflicts occur in scheduling.
80. Each classroom may host only one course-section per time slot.
81. Each faculty member may teach only one course per time slot.
82. Administrators can assign faculty, rooms, time slots, and seats to course sections.
83. Administrators can create new courses, departments, majors, minors, and prerequisites.

84. Administrators can cancel course sections (with automatic student notifications).
85. Course sections with fewer than five enrolled students must be dropped one week before the semester begins.
86. All administrative changes must be logged with:
    - Who made the change
    - What was changed
    - When it was changed

3.3.4. Explicitly Forbidden (Constraints)

87. Administration cannot assign full-time faculty to more than 2 course sections.
88. Administration cannot assign part-time faculty to more than 1 course section.
89. Administration cannot schedule two course sections in the same room or time slot.
90. Administration cannot alter user information without user request.
91. Administration cannot drop course sections after semester start.
92. Administration cannot assign faculty outside their department.
93. Administration cannot override the 5-student minimum for course-section continuation.
94. Administration cannot override seat limits or credit limits:
    - Full-time: ≤16 credits
    - Part-time: ≤8 credits
95. Administration cannot reuse CRNs for multiple course sections.
96. Administration cannot view or modify passwords.
97. Administration cannot create duplicate user IDs.
98. Administration cannot delete users (only deactivate or archive).
99. Administration cannot change or tamper with attendance records.
100. Administration cannot assign faculty to multiple course sections in the same time slot.
101. Administration with View-Only security cannot perform updates.
102. Administration cannot bypass prerequisites or enrollment eligibility rules.
103. Administration cannot enroll a student who has already passed a course.
104. Administration cannot change grade information without authorization.
105. Administration cannot modify faculty rosters or advisees without approval.

4. Use Cases

4.1. Student Use Cases

4.1.1. Unsuccessfully add a course-section

Use Case	Unsuccessfully add a course-section
Actors	Student
Initial Condition	● User is logged in and on the Registration page.
Flow of Events	1. User clicks on the “Add a course section” button. 2. User is presented with a list of course names. 3. User clicks on the course name. 4. User is presented with course-sections. 5. User clicks the course-section. 6. User is presented with a button to add the course-section. 7. User clicks add course section. 8. System detects issue and displays error message informing the student that they cannot add a course-

	section at this time due to one of the following reasons: a. The time window has passed to add a course-section b. Student has a hold on their account c. Student has already received a passing grade in this course-section d. There is a time conflict with another course-section they are registered for e. Student is an undergrad trying to add a graduate course f. Student is a graduate trying to add an undergraduate course g. Student exceeded 16 credits for full time or 8 credits for part time h. Student has not completed pre-requisite requirements. 9.

Exit Condition	User exits registration page

4.1.2. Successfully add a course-section

Use Case	Successfully add a course-section
Actors	Student
Initial Condition	• User is logged in and on the Registration page.
Flow of Events	1. User clicks on the “Add a course section” button. 2. User is presented with a list of course names. 3. User clicks on the course name. 4. User is presented with course-sections. 5. User clicks the course-section. 6. User is presented with a button to add the course-section. 7. User clicks add course section. 8. System displays message informing the user that they have added a course-section at this time.

Exit Condition	User exits registration page

4.1.3. Successfully drop a course-section

Use Case	Successfully drop a course-section
Actors	Student
Initial Condition	• User is logged in and on the Registration page.
Flow of Events	1. User clicks on the “Drop a course section” button. 2. User is presented with a list of course-sections student is currently enrolled in. 3. User clicks the course-section. 4. User is presented with a button to drop the course-section. 5. User clicks drop course section. 6. System displays message informing the student that they have dropped a course-section.
Exit Condition	User exits registration page

4.1.4. Unsuccessfully drop a course-section

Use Case	Unsuccessfully drop a course-section
Actors	Student
Initial Condition	● User is logged in and on the Registration page.
Flow of Events	1. User clicks on the “Drop a course section” button. 2. User is presented with a list of course-sections student is currently enrolled in. 3. User clicks the course-section. 4. User is presented with a button to drop the course-section. 5. User clicks drop course section. 6. System displays error message that the course could not be dropped (“Outside add/drop period”). 7. System detects issue and displays error message informing the student that they cannot add a course-section at this time due to one of the following reasons: a. Passed drop window
Exit Condition	User exits registration page

4.1.5. Successfully Declare Major

Use Case	Successfully declare major
Actors	Student
Initial Condition	● User is logged in and on the Registration page.
Flow of Events	1. Student goes to Info page 2. Student navigates to change major/minor section 3. Student selects change major 4. Complete form for changing major 5. Submit request
Exit Condition	User has successfully declared major

4.1.6. Successfully Declare Minor

Use Case	Successfully declare minor
Actors	Student
Initial Condition	● User is logged in and on the Registration page.
Flow of Events	1. Student goes to Info page 2. Student navigates to change major/minor section 3. Student selects change minor 4. Student completes form for changing minor 5. Student submits request

Exit Condition	Student has successfully declared minor

4.1.7. Unsuccessfully Declare Major

Use Case	Unsuccessfully declare major
Actors	Student
Initial Condition	● User is logged in and on the Registration page.
Flow of Events	1. Student goes to Info page 2. Student navigates to change major/minor section 3. Student selects change minor 4. Student completes form for changing minor 5. Student submits request 6. System displays message that major cannot be changed 7. System detects issue and displays message their major could not be changed due to: a) Technical Error b) Lacking a certain number of credits c) Trying to add more than 2 majors d) Already has major e) Adding major you already have f) Unsuccessfully changing major from graduate to undergraduate g) Unsuccessfully changing major from undergraduate to graduate
Exit Condition	Student exits Registration page

4.1.8. Unsuccessfully Declare Minor

Use Case	Unsuccessfully declare minor
Actors	Student
Initial Condition	● User is logged in and on the Registration page.
Flow of Events	1. Student goes to Info page 2. Student navigates to change major/minor section 3. Student selects change minor 4. Student completes form for changing minor 5. Student submits request 6. System displays message that minor cannot be changed 7. System detects issue and displays message their minor could not be changed due to: a) Technical Error b) Lacking a certain number of credits c) Have not been approved d) Already has minor e) Already has 2 majors f) Adding minor you already have
Exit Condition	Student exits Registration page

4.2. Faculty Use Cases

4.2.1. View Current Semester Schedule

Use Case	View Current Semester Schedule
Actors	Faculty/Admin
Initial Condition	● Faculty member is logged into system and on their faculty dashboard.
Flow of Events	1. The faculty member navigates to schedule from the dashboard. 2. The faculty member selects “Current Semester Schedule” 3. The system displays the current semester schedule including course names, CRNs, dates/times and building/room no. 4. Faculty views schedule details.
Exit Condition	Faculty member has successfully viewed their current semester schedule and may return to the dashboard or navigate to other functions.

4.2.2. View Next Semester Schedule

Use Case	View Next Semester Schedule
Actors	Faculty/Admin
Initial Condition	● User is logged into system and on their faculty dashboard.
Flow of Events	1. The faculty member navigates to schedule from the dashboard. 2. The faculty member selects “Next Semester Schedule” 3. The system displays the current semester schedule including course names, CRNs, dates/times and building/room no. 4. Faculty views schedule details.
Exit Condition	Faculty member has successfully viewed their next semester schedule and may return to the dashboard or navigate to other functions.

4.2.3. Successfully Assign Grades

Use Case	Successfully assign Grades
Actors	Faculty
Initial Condition	● User is authenticated (logged in). ● Faculty is assigned to selection course-section. ● Selected student is on class roster. ● User is viewing schedule ● Within one week of semester end
Flow of Events	1. The faculty member selects a course-section from their current semester schedule. 2. The system displays course-section details including class roster. 3. The faculty member enters final grades for each student in the section. 4. The system validates the format and completeness of grade entries. 5. The faculty submits grades for the course-section. 6. The system saves the grades, timestamps the submission, and writes action to log. 7. The system displays confirmation of submission.

Exit Condition	● Final grades are recorded for all students of a course-section. Submission time and faculty member are logged.

4.2.4. Unsuccessfully Assign Grades

Use Case	Unsuccessfully assign Grades
Actors	Faculty
Initial Condition	● User is authenticated (logged in). ● Faculty is assigned to selection course-section. ● Selected student is on class roster. ● User is viewing schedule ● Within one week of semester end
Flow of Events	1. The faculty member selects a course-section from their current semester schedule. 2. The system displays course-section details including class roster. 3. The faculty member enters final grades for each student in the section. 4. The system validates the format and completeness of grade entries. 5. The system displays an error message for one of the following reasons: a. Grades incomplete b. Invalid input type (must be A, A-, B+, B, B-, C+, C, C-, D+, D, D- or F c. Outside time window

Exit Condition	• Faculty corrects grading

4.2.5. Successfully Record Attendance

Use Case	Successfully record Attendance
Actors	Faculty
Initial Condition	● User is authenticated (logged in). ● Faculty is assigned to at least one course-section ● Selected student is on class roster
Flow of Events	1. The faculty member selects a course-section from their current semester schedule. 2. The system displays course-section details including class roster. 3. The faculty member selects class roster. 4. The system displays class roster. 5. The faculty submits attendance for students in their course-section. 6. The system validates the date of attendance. 7. The system saves attendance log. 8. The system displays confirmation of submission.
Exit Condition	● Attendance is recorded for all students of a course-section.

	Submission time and faculty member are logged.

4.2.6. Unsuccessfully Record Attendance

Use Case	Unsuccessfully record Attendance
Actors	Faculty
Initial Condition	● User is authenticated (logged in). ● Faculty is assigned to at least one course-section ● Selected student is on class roster
Flow of Events	1. The faculty member selects a course-section from their current semester schedule. 2. The system displays course-section details including class roster. 3. The faculty member selects class roster. 4. The system displays class roster. 5. The faculty submits attendance for students in their course-section. 6. The faculty member selects a course-section from their current semester schedule.

	7. The system displays course-section details including class roster. 8. The faculty member enters attendance for each student in the section. 9. The system validates the date of attendance. 10. The system displays an error message for one of the following reasons: a. Date of entry is before attendance date. b. Date of entry is after attendance date.
Exit Condition	● Faculty goes to todays date and records attendance.

Successfully Send Email Notifications to Students

Use Case	Successfully Send Email Notifications to Students
Actors	Faculty
Initial Condition	● The faculty member must be logged into the system ● Assigned to at least one course section.
Flow of Events	1. The faculty member selects “”Email” from Dashboard 2. The system displays Inbox emails of the faculty assigned email address. 3. The faculty member selects “Compose” from email screen. 4. The system displays a new email with inputs for recipient email address, subject and message. 5. The faculty member selects recipients from their course-section(s). 6. The system validates recipients. 7. The faculty member types subject and message. 8. The faculty member submits the email. 9. The system validates email delivery. 10. The system displays confirmation of sent email.

Exit Condition	● The recipients receive email

4.2.8. Unsuccessfully Send Email Notifications to Students

Use Case	Unsuccessfully Send Email Notifications to Students
Actors	Faculty
Initial Condition	● The faculty member must be logged into the system ● Assigned to at least one course section.
Flow of Events	1. The faculty member selects “”Email” from Dashboard 2. The system displays Inbox emails of the faculty assigned email address. 3. The faculty member selects “Compose” from email screen. 4. The system displays a new email with inputs for recipient email address, subject and message. 5. The faculty member selects recipients from their course-section(s). 6. The system validates recipients. 7. The faculty member types subject and message. 8. The faculty member submits the email.

	9. The system validates email delivery. 10. The system displays error message for one of the following reasons: a. Internet connection unstable b. Invalid recipient email c. Email too large
Exit Condition	● The faculty corrects error and resends message.

4.3. Administration Use Cases

4.3.1. Manage User Accounts(Student & Faculty)

Use Case	Manage User Accounts
Actors	Admin (Update security level)
Initial Condition	● Admin is authenticated with elevated privileges and has navigated to the *User Management* module
Flow of Events	1. Admin selects create New User Account or searches and existing one. 2. Admin chooses account type: Student or Faculty. 3. For Student, Admin enters program, enrollment year, status (active, leave, graduated). 4. For faculty: Admin enters department, role (lecturer, professor) and employment status. 5. Admin updates details(e.g., course assignments for faculty, major for students). 6. Admin deactivates/ archives accounts as needed (graduation, resignation, contract end). 7. System validates, stores update, sending confirmation to admin.

Exit Condition	● Account updates are reflected in real time across all modules (course registration, grading, billing).

4.3.2. Course Management (Create | Update | Delete | Courses)

Use Case	Course Management
Actors	Admin (Update security level)
Initial Condition	● Admin is authenticated and has course management rights.
Flow of Events	1. Admin selects *Manage Courses* from portal menu. 2. Admin creates a new course entry (course Code, Title, Credits, Department, Instructor). 3. Admin updates course details (Schedule, pre-requisites, max enrollment). 4. Admin can deactivate/remove outdated or duplicate courses. 5. System logs changes and notifies relevant faculty and students of updates.

Exit Condition	● Course catalog is synchronized for all admin, student and faculty views.

4.3.3. Enrollments section

Use Case	Enrollment Oversight
Actors	Admin (Update security level)
Initial Condition	● Admin is authenticated and has course management rights. ● Enrollment period is open
Flow of Events	1. Admin opens the Enrollment Dashboard. 2. Admin monitors active enrollment, waitlist, and capacity thresholds. 3. Admin adjusts capacity for courses based on demand or departmental rules. 4. Admin intervenes in exceptional cases (manual enrollment or withdrawal). 5. System update records, ensuring no violation of pre-requisite or credit limits.
Exit Condition	● Enrollment Data is finalized and reflected in the central database.

4.3.4. System security & Access Control

Use Case	System Security and Access Control
Actors	Admin (Update security level)
Initial Condition	● Admin is authenticated and Update security level ● Enrollment period is open
Flow of Events	1. Admin navigates to Security Settings. 2. Admin reviews access logs and suspicious login attempts. 3. Admin resets credentials or unlocks accounts flagged by intrusion detection. 4. Admin set role based access controls for staff, faculty and students 5. System updates policies in real time propagating across all modules.
Exit Condition	● System ensures integrity, compliance with institutional IT policy, and audit logs are preserved.

4.3.5. Successfully Cancel course Section

Use Case	Successfully Cancel course Section
Actors	Admin
Initial Condition	Course Section has 5 or fewer students and cancellation request is more than a week before semester begins.
Flow of Events	1. Admin selects course section to cancel. 2. System validates enrollment count and time window 3. If valid, course section is canceled. 4. Student and and faculty is notified automatically.
Exits Condition	Course Section is removed from active schedule.

4.3.6. Unsuccessfully Cancel Course Section

Use Case	Successfully Cancel course Section
Actors	Admin
Initial Condition	Course section has more than 5 student or cancellation requests is less than a week before semester starts.
Flow of Events	1. Admin request cancellation 2. The system checks rules (time window, enrollment count). 3. if not satisfied, cancellation is blocked. 4. The system display ruled error message.
Exits Condition	Course section remains active.

4.3.7. Successful Pre-Semester Update(Room/Faculty Change)

Use Case	Successful Pre-Semester Update(Room/Faculty Change)
Actors	Admin
Initial Condition	Semester has not started.
Flow of Events	1. Admin selects Course Section 2. Admin updates faculty or room 3. System validates availability and time conflicts 4. Changes are saved 5. Notification is sent to student and faculty.
Exits Condition	Course Section updated Successfully in System

4.3.8. Unsuccessful Pre-Semester updates (Room/Faculty Change)

Use Case	Unsuccessful Pre-Semester updates (Room/Faculty Change)
Actors	Admin
Initial Condition	Admin attempts to update course-section assignments
Flow of Events	1. Admin selects Course Section. 2. Admin attempts to update faculty or room. 3. System validates information 4. System displays error for one of the following reasons: a. Time conflict b. Room conflict c. Faculty outside department d. Faculty already assigned to maximum courses.
Exits Condition	Course section remains unchanged

4.3.9. Successfully create Course

Use Case	Successfully create Course
Actors	Admin
Initial Condition	Admin logged in with course creation privileges.
Flow of Events	1. Admin navigates to Create Course. 2. Admin enters course details:CRN,Course name, section, faculty, time slot, building/room, department, course-prerequisites. 3. System validates the following rules a. Faculty load(Full-time 2 or less; Part-time 1 or less) b. No duplicate CRN or section c. No room conflicts d. No time conflicts e. The faculty belongs to the department. 4. A valid Course is created.
Exits Condition	Course appears in Catalog and is available for scheduling and enrollment.

4.3.10. Unsuccessfully Create Course

Use Case	Unsuccessfully Create Course
Actors	Admin
Initial Condition	Admin attempts to create a course.
Flow of Events	1. Admin enter course details 2. System validates the rules 3. If violated any of these rules: creation is blocked a. Faculty overload b. Duplicate CRN c. Duplicate section d. Room conflict e. time conflict f. Faculty outside of the department. 4. Error message is shown or displayed
Exits Condition	No Course Created; Catalog unchanged.

4.3.11. Successfully Update Course

Use Case	Successfully Update Course
Actors	Admin
Initial Condition	Course already exists in catalog.
Flow of Events	1. Admin navigates to update Course 2. Admin modifies details(CRN, section,faculty assignment, time slot, room) 3. System validates rules: a. Faculty load and department match b. No duplication of CRN/ section c. No time/room conflicts 4. If valid, update saved.
Exits Condition	Course updated and changes made are reflected in schedule and enrollment.

4.3.12. Unsuccessfully Update Course

Use Case	Unsuccessfully Update Course
Actors	Admin
Initial Condition	Admin attempts to update Course Details
Flow of Events	1. Admin edits course details 2. System validates rules 3. If rules fail (outside of update window, invalid faculty load,duplicate course,room and time conflict) the update is blocked. 4. Error message displayed.
Exits Condition	Course remains unchanged in the Catalog.

4.3.13. Successfully Update a student's Grade in Course-Section

Use Case	Successfully update a student's Grade in Course-Section
Actors	Admin
Initial Condition	Students record's exists, and the course-section is completed.
Flow of Events	1. Admin searches student and course-section. 2. Admin inputs the new grade. 3. Admin submit the update. 4. The system validates permissions and data. 5. System update grade in the record.
Exits Condition	Student's grade is successfully updated

4.3.14. Unsuccessfully update a student's Grade in Course-Section

Use Case	Unsuccessfully update a student's Grade in Course-Section
Actors	Admin
Initial Condition	Students record's exists, and the course-section is completed.
Flow of Events	1. Admin attempts to update a grade. 2. Admin inputs invalid data or lacks permission. 3. Admin submits the update. 4. The system checks permission and data validity. 5. The system blocks the update. 6. An error message pops up indicating failure for one of the following reasons: a. Course incomplete b. Invalid grade format.
Exits Condition	Student's grade remains unchanged, and an error message is shown.

4.3.15. Successfully Assign Time Slots & Rooms

Use Case	Successfully Assign Time Slots & Rooms
Actors	Admin
Initial Condition	Course Scheduling window is open.
Flow of Events	1. Admin selects courses to schedule. 2. Admin assigns available time slot and rooms 3. System checks for conflicts and availability. 4. System saves the assignments,
Exits Condition	Courses are assigned to specific time slot and rooms successfully with no conflicts.

4.3.16. Unsuccessfully Assign Time Slots & Rooms

Use Case	Unsuccessfully Assign Time Slots & Rooms
Actors	Admin
Initial Condition	Course Scheduling window is open.
Flow of Events	1. Admin selects course-section to assign faculty/room 2. Admin assigns available time slot and rooms 3. System checks for conflicts and availability. 4. System displays error message for one of the following reasons: a. Time conflict b. Room conflict c. Faculty already assigned to maximum number of course sections
Exits Condition	Course-section remains unassigned.

4.3.17. Successfully create Master Schedule Entry

Use Case	Successfully create Master Schedule.
Actors	Admin
Initial Condition	Master Schedule entry does not exist

Flow of Events	1. Admin selects the schedule to create. 2. Admin inputs schedule details (CRN, course-section, course name, timeslot, building/rooms, faculty, credits). 3. System validates the input data for conflicts and completeness 4. System saves the schedule.
Exits Condition	Master Schedule is successfully created

4.3.18. Successfully Update Master Schedule Entry

Use Case	Successfully Update Master Schedule.
Actors	Admin
Initial Condition	Master Schedule entry exists.
Flow of Events	1. Admin selects the schedule to update. 2. Admin selects course-section to update 3. Admin updates schedule details (CRN, course-section, course name, timeslot, building/rooms, faculty, credits). 4. System validates the input data for conflicts and completeness 5. System saves the schedule.
Exits Condition	Master Schedule is successfully updated

4.3.19. Unsuccessfully create Master Schedule Entry

Use Case	Unsuccessfully create Master Schedule Entry
Actors	Admin
Initial Condition	Admin attempts to create Master Schedule Entry
Flow of Events	1. Admin selects the schedule to create. 2. Admin inputs schedule details (CRN, course-section, course name, timeslot, building/rooms, faculty, credits). 3. System validates the input data for conflicts and completeness 4. System displays error message for one of the following reasons: a. Invalid details b. Time conflict c. Room Conflict d. Duplicate entry
Exits Condition	Master Schedule remains unchanged.

4.3.20. Unsuccessfully Update Master Schedule Entry

Use Case	Unsuccessfully Update Master Schedule Entry
Actors	Admin

Initial Condition	Admin attempts to create Master Schedule Entry
Flow of Events	1. Admin selects the schedule to update. 2. Admin selects course-section to update 3. Admin updates schedule details (CRN, course-section, course name, timeslot, building/rooms, faculty, credits). 4. System validates the input data for conflicts and completeness 5. System displays error message for one of the following reasons: a. Invalid details b. Time conflict c. Room Conflict d. Duplicate entry
Exits Condition	Master Schedule remains unchanged.

4.3.21. Successfully Create Department Major/Minor

Use Case	Successfully Update Department Major/Minor
Actors	Admin
Initial Condition	Major/Minor does not exist in system
Flow of Events	1. Admin selects department.

	2. System displays list of department majors/minors. 3. Admin selects create new major/minor. 4. Admin inputs major/minor information a. Name b. Department 5. System validates major/minor information. 6. System saves the information.
Exits Condition	Department major/minor is successfully created.

4.3.22. Successfully Update Department Major/Minor

Use Case	Successfully Update Department Major/Minor
Actors	Admin
Initial Condition	Major/Minor exists in system
Flow of Events	1. Admin selects department. 2. System displays list of department majors/minors. 3. Admin selects major/minor. 4. Admin inputs major/minor update information a. Name b. Department 5. System validates major/minor information. 6. System saves the information.

Exits Condition	Department major/minor is successfully updated.

4.3.23. Unsuccessfully Create Department Major/Minor

Use Case	Unsuccessfully Update Department Major/Minor
Actors	Admin
Initial Condition	Major/Minor does not exist in system
Flow of Events	1. Admin selects department. 2. System displays list of department majors/minors. 3. Admin selects create new major/minor. 4. Admin inputs major/minor information a. Name b. Department 5. System validates major/minor information. 6. System displays error message for one of the following reasons: a. Outside time window b. Faculty assigned too many courses (1 (PT) or 2 courses (FT)) c. Room conflict d. Time conflict e. Duplicate section f. Duplicate CRN g. Faculty outside department

Exits Condition	Department major/minor is unsuccessfully created.

4.3.24. Unsuccessfully Update Department Major/Minor

Use Case	Unsuccessfully Update Department Major/Minor
Actors	Admin
Initial Condition	Major/Minor exists in system
Flow of Events	1. Admin selects department. 2. System displays a list of department majors/minors. 3. Admin selects major/minor. 4. Admin inputs major/minor update information a. Name b. Department 5. System validates major/minor information. 6. System displays error message for one of the following reasons: a. Outside time window b. Faculty assigned too many courses (1 (PT) or 2 courses (FT)) c. Room conflict d. Time conflict e. Duplicate section f. Duplicate CRN

	g. Faculty outside department
Exits Condition	Department major/minor is unsuccessfully updated.

4.3.25. Successfully Create Department

Use Case	Successfully Update Department
Actors	Admin
Initial Condition	Department does not exist in system
Flow of Events	1. Admin selects Create New Department.. 2. Admin inputs department information a. Name b. Graduate/Undergraduate 3. System validates department information. 4. System saves the information.
Exits Condition	Department is successfully created.

4.3.26. Unsuccessfully Create Department

Use Case	Unsuccessfully Update Department
Actors	Admin
Initial Condition	Attempts to create a new department
Flow of Events	1. Admin selects Create New Department.. 2. Admin inputs department information a. Name

	b. Graduate/Undergraduate 3. System validates department information. 4. System displays error message for one of the following reasons: a. Department already exists b. Invalid name
Exits Condition	Department list remains unchanged.

4.3.27. View All Users

Use Case	View All Users
Actors	Admin
Initial Condition	Admin is authenticated (logged in). Admin has appropriate security level (View-Only or Update). User records (Students, Faculty, Admins) exist in the system.
Flow of Events	1. Admin navigates to **Administration → Users → View All Users**. 2. The system displays a searchable, filterable list of users (name, user UID, role, status, email). 3. Admin applies optional filters (Role: Student/Faculty/Admin; Status: Active/Inactive;

	Department; Access Level) and/or enters a search term. 4. The system refreshes the results with pagination and sorting (e.g., by name, userID, role). 5. Admin selects a user to view the profile. 6. The system displays the user's profile details (read-only), excluding any passwords or password hashes. 7. If Admin has **Update** access and chooses to proceed, the system provides navigation to separate update actions (e.g., deactivate/activate account, update contact info, assign roles/departments) without changing data in this view.
Exits Condition	Admin has successfully viewed the list of users. Admin with update access can update individual profiles.

4.3.28. Create Course Prerequisite

Use Case	Create Course Prerequisite
Actors	Admin
Initial Condition	Admin is authenticated (logged in).

	Admin has **Update** security level. Target **course** exists in the catalog. Candidate **prerequisite course(s)** exist in the catalog.
Flow of Events	1. Admin navigates to Master Schedule 2. The System displays master schedule 3. Admin selects course. 4. System displays course information 5. Admin selects add prerequisite 6. System displays list of courses 7. Admin selects courses to list as target course prerequisites
Exits Condition	Prerequisite(s) are created and visible on the target course

4.3.29. Update Course Prerequisite

Use Case	Update Course Prerequisite
Actors	Admin
Initial Condition	Admin is authenticated (logged in). Admin has **Update** security level.

	Target **course** exists and has an existing prerequisite set
Flow of Events	1. Admin navigates to Master Schedule 2. The System displays master schedule 3. Admin selects course. 4. System displays course information including existing prerequisites 5. Admin selects prerequisite and clicks remove prerequisite 6. System updates target course prerequisite
Exits Condition	Prerequisite(s) are updated and visible on the target course

4.4. Multi-User Use Cases

4.4.1. View Master Schedule

Use Case	View Master Schedule
Actors	Students/Faculty/Admin
Initial Condition	● User is authenticated
Flow of Events	1. User selects “Master Schedule” 2. The system retrieves and displays the list of offered courses 3. User applies search filters (semester department, CRN, course name, time slot, building/room no., assigned faculty, credits or available seats.) 4. System applies selected search filters and displays results. 5. User browses the Master Schedule results.
Exit Condition	● User successfully viewed Master Schedule in read-only and can return to dashboard or navigate to functions.

4.4.2. View Degree Progress

Use Case	View Advisee Degree Progress

Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated (logged in). ● Target student record exists. ● (Faculty) Target student is on the faculty member's **roster** or **advisee** list. ● (Admin) User has appropriate security level (**View-Only** or **Update**). ● Grades for the chosen term have been posted.
Flow of Events	1. User selects to Student Information from their Dashboard or from a class roster/Advisee list. 2. The system retrieves selected student information. 3. User selects term from Degree Progress 4. The system displays degree progress 5. The user reviews degree progress a. Courses still needed b. Courses completed c. Course prerequisites d. Department e. Major/Minor
Exit Condition	● The advisee's degree progress is displayed (read-only); ● the faculty member can navigate to course offerings or return to the advisee overview.

4.4.3. View Advisee List

Use Case	View Advisee List
Actors	Faculty/Admin
Initial Condition	● User is authenticated (logged in). ● Faculty must be assigned an advisee
Flow of Events	1. User selects Advisees from the dashboard. 2. The system retrieves and displays the Advisee List (names, IDs, program level/major). 3. The faculty member scrolls, filters, or searches advisees as needed. 4. Change/Administrative Request form.
Exit Condition	● The Advisee List is displayed; the user may open an advisee's record or return to the dashboard.

4.4.4. Attempts to change student information

Use Case	Attempt to Change student information
Actors	Student/Faculty/Admin (view-only)
Initial Condition	● User is authenticated (logged in). ● Target student record exists. ● (Faculty) Target student is on the faculty member's **roster** or **advisee** list.
Flow of Events	1. User opens Student Information a. Students from dashboard b. Faculty from advisee list/class roster c. Admin from 2. The system displays read-only student information (schedules, grades, unofficial transcript, major/minor, holds). 3. User attempts to modify protected data (e.g., change grade, edit schedule, edit personal info). 4. The system evaluates permissions and context (not the grade window; faculty not authorized for changes). 5. The system blocks the edit and logs the attempt (who/what/when). 6. The system shows policy guidance: "Read-only. Edits require approved administrative workflow," and may link to a Grade Change/Administrative Request form.

Exit Condition	● No schedule data is changed; the denial is shown and the attempt is audit-logged. Faculty and Admin may continue allowed (read-only) actions.

4.4.5. Successfully View Course Prerequisites

Use Case	Successfully View Course Prerequisites
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated (logged in). ● User is viewing master schedule
Flow of Events	1. User selects term 2. System retrieves and displays course listing information for selected term 3. User selects course name 4. System retrieves and displays course information including a. Time slot b. Building/room no. c. Faculty d. Department e. Credits f. Course Name g. CRN h. Prerequisites i. Available seats

Exit Condition	● Master schedule is displayed correctly ● Course prerequisites is displayed correctly

4.4.6. Unsuccessfully View Course Prerequisites

Use Case	Unsuccessfully View Course Prerequisites
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated (logged in). ● User is viewing master schedule
Flow of Events	1. User selects term 2. System retrieves and displays course listing information for selected term 3. User selects course name 4. System displays error message with “No Course Prerequisites”
Exit Condition	● User searches for another course or exits master schedule.

4.4.7. View Class Roster

Use Case	View Class Roster
Actors	Faculty/Admin
Initial Condition	● User is logged into system and on their faculty dashboard. ● Faculty is the assigned instructor. ● User is on display page for course-section.
Flow of Events	1. Faculty opens class roster from My Schedule on Dashboard 2. Admin views from Master Schedule. 3. User selects a term. 4. System verifies permissions 5. The system retrieves and displays roster info including Student names, Student ID, enrollment status, email, Major/Minor 6. User reviews roster. 7. System logs who view course section roster and when
Exit Condition	The class roster is displayed correctly

4.4.8. Attempt to Change Schedule (Forbidden)

Use Case	Attempt to Change Schedule
Actors	Student/Faculty/Admin (view-only)
Initial Condition	● User must be logged into the system
Flow of Events	1. User opens Present/Next Semester Schedule. 2. The system displays course-sections (course, CRN, time slot, building/room). a. Student enrolled course-section b. Faculty assigned course-sections 3. User attempts to edit a meeting time or building/room (e.g., clicks an edit control or accesses a deep link). 4. The system checks permissions and detects that schedule edits are Admin-only. 5. The system denies the change (no edit controls or returns a clear error), and logs the attempt in the audit log. 6. The system displays a policy message: “Schedules can only be updated by Administration with Update Level access.”
Exit Condition	● No schedule data is changed; the denial is shown and the attempt is audit-logged. Faculty and Admin

	may continue allowed (read-only) actions.

4.4.9. Unsuccessful Login/Password Reset

Use Case	Log in (unsuccessful login - password reset)
Actors	Student/Faculty/Admin
Initial Condition	User has valid login credentials (UserID and password) and the system is online and accessible through the login page.
Flow of Events	<u>Flow of Events</u>: 1. User attempts to log into the system. 2. The credentials are invalid (3 attempts). 3. User account is locked out, reset button. 4. User will get a generated code on their school email associated with the account to reset password. 5. User will have to enter the code to reset the password. 6. User will enter a new password and use

	that password to log in. 7. User will be able to log into the system with the new password.
Exit Condition	User will log out and exit the system.

4.4.10. Successful Login

Use Case	Successful Login
Actors	Student/Faculty/Admin
Initial Condition	User has valid login credentials (UserID and password) and the system is online and accessible through the login page.
Flow of Events	<u>Flow of Events</u>: 1. User attempts to log into the system. 2. The credentials are valid. 3. User will log into the system with their username and password.
Exit Condition	The user will be presented with their dashboard.

4.4.11. View Student Info

Use Case	View Student Info
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated and on their dashboard. ● Student record exists in the system. ● (Faculty) The student is on their class roster or advisee list. ● (Admin) User has appropriate security level (View-Only or Update).
Flow of Events	1. User selects **"Student Information"** from their dashboard or from a linked roster/advisee list. 2. System retrieves the requested student record. 3. System displays the following details: a. Student profile (name, ID, major/minor, department, standing) b. Current, past, and next semester schedules c. Grades and semester GPA

	d. Cumulative GPA and unofficial transcript e. Degree audit / progress toward major/minor requirements f. Course prerequisites (linked from requirements view) g. Holds (Financial, Health, Academic) h. degree progress 4. User reviews the displayed information. a. **Student**: Sees their own information only. b. **Faculty**: Sees advisee or roster student's information in **read-only mode**. c. **Admin with View-Only**: Sees any student's information in **read-only mode**. d. **Admin with Update access**: May also update fields such as address, phone, program assignment, or schedule overrides (within institutional rules). 5. System logs the access (who viewed, what was viewed/changed, when).

Exit Condition	● Student information is displayed successfully. ● For Admin with Update access, authorized changes are saved and logged. ● No unauthorized changes occur; system enforces read-only restrictions.

4.4.12. View Semester Grades

Use Case	View Semester Grades
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated (logged in). ● Target student record exists. ● (Faculty) Target student is on the faculty member's **roster** or **advisee** list. ● (Admin) User has appropriate security level (**View-Only** or **Update**). ● Grades for the chosen term have been posted.
Flow of Events	1. User selects to Student Information from their Dashboard or from a class roster/Advisee list. 2. The system retrieves selected student information. 3. User selects term from Semester Grades view 4. The system displays grades for requested student for a selected semester 5. The user reviews grades a. Student sees their own grades b. Faculty sees roster/advisee students grades in view only.

	c. Admin views requested students grades.
Exit Condition	● The posted semester grades are displayed ● No unauthorized changes occurred

4.4.13. View GPA and Unofficial Transcript

Use Case	View Unofficial Transcript
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated (logged in). ● Target student record exists. ● (Faculty) Target student is on the faculty member's **roster** or **advisee** list. ● (Admin) User has appropriate security level (**View-Only** or **Update**). ● Student has taken at least one course
Flow of Events	1. User navigates to Unofficial Transcript from Student Information. 2. The system retrieves and displays requested students unofficial transcript

	3. User reviews the following information from Unofficial Transcript a. Overall GPA b. Course names c. Course grades
Exit Condition	Student has viewed transcript

4.4.14. View Degree Audit

Use Case	View Unofficial Transcript
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated (logged in). ● Target student record exists. ● (Faculty) Target student is on the faculty member's **roster** or **advisee** list. ● (Admin) User has appropriate security level (**View-Only** or **Update**). ● Student has taken at least one course
Flow of Events	1. User navigates to Degree Audit from Student Information.

	2. System obtains and displays degree info a. Major/minor b. Course names c. Overall GPA d. Course Grade 3. User views Degree Audit
Exit Condition	Student has degree audit

4.4.15. View holds

Use Case	View Semester Holds
Actors	Student/Faculty/Admin
Initial Condition	● User is logged into system ● (Faculty) Target student is on the faculty member's **roster** or **advisee** list. ● (Admin) User has appropriate security level (**View-Only** or **Update**). ● Student has taken at least one course ● User is viewing student information

Flow of Events	1. User selects View Holds from Student Information 2. The system displays holds on student accounts. 3. The user views requested students account holds.
Exit Condition	User exits Holds section

4.4.16. Successfully Reset Password

Use Case	Successfully Reset Password
Actors	Student/Faculty/Admin
Initial Condition	● User account exists in the system. ● User has selected “Forgot/Reset Password.” ● User has access to their registered email.
Flow of Events	1. User requests password reset. 2. The system prompts for User ID or email. 3. The system generates and sends a reset link/code to the user’s registered email. 4. User opens the link or enters the code. 5. User creates a new compliant password. 6. The system updates the password and confirms reset.
Exit Condition	User has successfully reset their password and can log in with the new credentials.

4.4.17. Change Password

Use Case	Change Password
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated (logged in).
Flow of Events	1. User navigates to profile/security settings. 2. User enters current password and new password. 3. The system validates the current password. 4. The system validates the new password against password policy. 5. The system updates the password and confirms success.
Exit Condition	Password is changed and user can log in with the new password.

4.4.18. View Department

Use Case	View Department
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated. ● Department exists in the system.
Flow of Events	1. User selects a department from Master Schedule. 2. The system retrieves and displays department information. 3. User reviews department details.
Exit Condition	Department details are displayed.

4.4.19. View Courses of Department

Use Case	View Courses of Department
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated. ● Department exists with courses.
Flow of Events	1. User selects a department from Master Schedule. 2. The system retrieves and displays department information. 3. User selects “Courses” within a department. 4. The system retrieves and displays all courses in that department. 5. User reviews the course list.
Exit Condition	Department’s courses are displayed.

4.4.20. View Majors/Minors of Department

Use Case	View Majors/Minors of Department
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated. ● Department exists with courses.
Flow of Events	1. User selects a department from Master Schedule. 2. The system retrieves and displays department information. 3. User selects “Majors/Minors” within a department. 4. The system retrieves and displays all majors/minors in that department. 5. User reviews the list of majors and minors
Exit Condition	Department’s majors/minors are displayed.

4.4.21. View Majors/Minors Course Requirements

Use Case	View Majors/Minors of Department
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated. ● Major exists with defined requirements.
Flow of Events	1. User selects a Majors/Minors from Master Schedule. 2. The system retrieves and displays Major/Minor information. 3. User selects a major/minor. 4. The system retrieves and displays the major's/minor's requirements. 5. User reviews required and elective courses.
Exit Condition	Major/Minor course requirements are displayed.

4.4.22. View Schedule for Fall 2025

Use Case	View Schedule for Fall 2025
Actors	Student/Faculty/Admin
Initial Condition	User is authenticated. Fall 2025 schedule is published. Role rules: ● Student: may only view own schedule. ● Faculty: may view rosters or advisees. ● Admin: may view any student.
Flow of Events	1. Student selects Fall 2025 schedule from their schedule. 2. Faculty selects Fall 2025 from class roster/advisee list. 3. Admin selects Fall 2025 from Student Information. 4. The system retrieves schedule data for the target student. 5. The system enforces role permissions. 6. The system displays enrolled course-sections.
Exit Condition	Fall 2025 schedule is displayed according to role permissions.

4.4.23. View Schedule for Spring 2026

Use Case	View Schedule for Spring 2026
Actors	Student/Faculty/Admin
Initial Condition	User is authenticated. Spring 2026 schedule is published. Role rules: ● Student: may only view own schedule. ● Faculty: may view rosters or advisees. ● Admin: may view any student.
Flow of Events	1. Student selects Spring 2026 schedule from their schedule. 2. Faculty selects Spring 2026 from class roster/advisee list. 3. Admin selects Spring 2026 from Student Information. 4. The system retrieves schedule data for the target student. 5. The system enforces role permissions. 6. The system displays enrolled course-sections.
Exit Condition	Spring 2026 schedule is displayed according to role permissions.

4.4.24. Successfully Search for Advisor's Info

Use Case	Successfully Search for Advisor's Info
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated. ● Advisor records exist.
Flow of Events	1. User searches for advisor by student or advisor name/ID. 2. The system retrieves the matching advisor. 3. The system displays advisor contact information.
Exit Condition	Advisor information is displayed.

4.4.25. Unsuccessfully Search for Advisor's Info

Use Case	Unsuccessfully Search for Advisor's Info
Actors	Student/Faculty/Admin
Initial Condition	● User is authenticated. ● User has entered advisor search criteria.
Flow of Events	1. User searches for advisor. 2. The system attempts to retrieve advisor information. 3. The system cannot find a match. 4. The system displays "Advisor not found."
Exit Condition	User is notified advisor could not be found.

4. Entity/Relationships

*****Many to Many Relationships require an Associative Entity to separate the M to M relationship**

User

- UserID PRIMARY KEY, UNIQUE #(5-6 random digits)
- FirstName
- MiddleName
- LastName
- Street Address (House Number, Stree)
- City
- State
- Zipcode
- Gender (‘M’ OR ‘F’)
- DOB
- UserType (‘Student’, ‘Faculty’, ‘Admin’)
- Email
- Status

Login

- LoginID PRIMARY KEY
- UserID FOREIGN KEY (UserID) REFERENCES Users(UserID)
- UserType
- Password
- LoginAttempts
- FOREIGN KEY (LoginID) REFERENCES Users(UserID)

Student

- StudentID PRIMARY KEY
- MajorID FOREIGN KEY (majorID) REFERENCES Major(MajorID)
- MinorID FOREIGN KEY (minorID) REFERENCES Minor(MinorID)

- StudentType
- FOREIGN KEY (studentID) REFERENCES Users(userID)

<u>UndergradStudent</u>

- studentID PRIMARY KEY
- DeptID FOREIGN KEY (deptID) REFERENCES Department(DeptID)
- UndergradStudentType
- FOREIGN KEY (studentID) REFERENCES Student(StudentID)

<u>FullTIme Undergrad</u>

- StudentID PRIMARY KEY
- Year
- MaxCredits
- MinCredits
- CreditsEarned
- FOREIGN KEY (StudentID) REFERENCES Student(StudentID)

<u>PartTimeUndergrad</u>

- StudentID PRIMARY KEY
- Year
- MaxCredits
- MinCredits
- CreditsEarned
- FOREIGN KEY (StudentID) REFERENCES Student(StudentID)

<u>Graduate Student</u>

- StudentID PRIMARY KEY
- DeptID FOREIGN KEY (DeptID) REFERENCES Dept(DeptID)

- Program (PhD, MS)
- GraduateStudentType
- FOREIGN KEY (StudentID) REFERENCES Student(StudentID)

<u>FullTimeGrad</u>

- StudentID PRIMARY KEY
- Year
- Credits Earned
- ThesisYear
- FOREIGN KEY (StudentID) REFERENCES Student(StudentID)

<u>PartTimeGrad</u>

- StudentID PRIMARY KEY
- Year
- Credits Earned
- ThesisYear
- FOREIGN KEY (StudentID) REFERENCES Student(StudentID)

<u>StudentHistory</u>

- StudentID FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
- CRN FOREIGN KEY (CRN) REFERENCES CourseSection(CRN)
- CourseID FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
- SemesterID FOREIGN KEY (SemesterID) REFERENCES Semester(SemesterID)
- Grade FOREIGN KEY (Grade) REFERENCES StudentEnrollment(Grade)
- PRIMARY KEY (StudentID, CRN)

Hold

- HoldID PRIMARY KEY
- HoldType

StudentHold

- StudentID FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
- HoldID FOREIGN KEY (HoldID) REFERENCES Hold(HoldID)
- DateOfHold
- PRIMARY KEY (StudentID, HoldID)

Faculty

- FacultyID PRIMARY KEY,
- OfficeID FOREIGN KEY (OfficeID) REFERENCES Room(RoomID)
- Specialty
- Rank
- FacultyType
- FOREIGN KEY (FacultyID) REFERENCES Users(UserID)

Admin

- AdminID PRIMARY KEY
- SecurityType
- FOREIGN KEY (AdminID) REFERENCES Users(UserID)

Department (1 to 1 with Faculty) - Associative Entity (Chair (1 to 1 with faculty, 01 with Dept)

- DeptID PRIMARY KEY,
- DeptName
- Email
- Phone
- RoomID FOREIGN KEY REFERENCES Room(RoomID)
- ChairID FOREIGN KEY (ChairID) REFERENCES Chair(ChairID)

- DeptAssistant

Faculty_Dept (Associative entity between department and faculty

- FacultyID FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
- DeptID FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
- %Time
- DOA (date of assignment)
- PRIMARY KEY (FacultyID, DeptID)

Chair

- ChairID PRIMARY KEY,
- FacultyID
- FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)

ViewAdmin

- AdminID PRIMARY KEY,
- FOREIGN KEY (AdminID) REFERENCES Faculty(FacultyID)

UpdateAdmin

- AdminID PRIMARY KEY,
- FOREIGN KEY (AdminID) REFERENCES Faculty(FacultyID)

FullTimeFaculty

- FacultyID PRIMARY KEY
- MinCourses 0
- MaxCourses 2
- FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)

PartTimeFaculty

- FacultyID PRIMARY KEY
- MinCourses 0
- MaxCourses 1
- FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)

Course

- CourseID PRIMARY KEY,
- CourseName
- DeptID FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
- Course_Desc
- NumCredits
- CourseType (grad/undergrad)

CoursePrerequisite (M to M with itself (course)) - Recursive Relationship

- CourseID FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
- PrerequisiteCourseID FOREIGN KEY (PrerequisiteCourseID) REFERENCES Course(CourseID)
- MinGradeRequired
- DecisionDate
- PRIMARY KEY (CourseID, PrerequisiteCourseID)

StudentEnrollment (between Student and CourseSection)

- StudentID FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
- CRN FOREIGN KEY (CRN) REFERENCES CourseSection(CRN)
- EnrollmentDate
- Grade
- PRIMARY KEY (StudentID, CRN)

Course-Section (Class)

- CRN PRIMARY KEY,
- CourseID FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
- CourseSectionNo
- FacultyID FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
- TimeSlotID FOREIGN KEY (TimeSlotID) REFERENCES (TimeSlot(TS_ID)
- RoomID FOREIGN KEY (RoomID) REFERENCES (Room(RoomID)
- Year
- SemesterID FOREIGN KEY (SemesterID) REFERENCES Semester(SemesterID)
- AvailableSeats

StatStaff

- StatStaffID PRIMARY KEY
- Status (active/inactive)
- FOREIGN KEY (StatStaffID) REFERENCES Users(UserID)

Timeslot

- TS_ID PRIMARY KEY,
- DayID FOREIGN KEY (DayID) REFERENCES (Day(DayID))
- PeriodID FOREIGN KEY (PeriodID) REFERENCES (Period(PeriodID))

- DayID FOREIGN KEY (DayID) REFERENCES (Day(DayId)

TimeSlotDay

- TS_ID FOREIGN KEY (TS_ID) REFERENCES (Timeslot(TS_ID)
- DayID FOREIGN KEY (DayID) REFERENCES (Day(DayID)
- PRIMARY KEY (TS_ID, DayID)

Day (M to M relationship with Timeslot)

- DayID PRIMARY KEY,
- DayOfWeek
- TS_ID FOREIGN KEY (TS_ID) REFERENCES Timeslot(TS_ID)
- PeriodID

TimeSlotPeriod

- TS_ID FOREIGN KEY (TS_ID) REFERENCES (Timeslot(TS_ID)
- PeriodID FOREIGN KEY (PeriodID) REFERENCES (Period(PeriodID)
- PRIMARY KEY (TS_ID, PeriodID)

Period (M to M relationship with Timeslot)

- PeriodID PRIMARY KEY,
- StartTime
- EndTime

Semester

- SemesterID PRIMARY KEY,
- SemesterName (fall/spring)
- Year
- StartDate
- EndDate
- AddDropDeadline

CourseSectionAttendance

- StudentID FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
- CRN FOREIGN KEY (CRN) REFERENCES CourseSection(CRN)
- CourseID FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
- AttendanceDate
- AttendancePeriod
- Present/Absent
- PRIMARY KEY (StudentID, CRN)

FacultyHistory

- CRN FOREIGN KEY (CRN) REFERENCES CourseSection(CRN)
- FacultyID FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
- CourseID FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
- SemesterID FOREIGN KEY (SemesterID) REFERENCES Semester(SemesterID)
- PRIMARY KEY (FacultyID, CRN)

Building

- BuildingID PRIMARY KEY,
- BuildingName
- Usage

Room - SuperType

- RoomID PRIMARY KEY,
- RoomNo
- BuildingID FOREIGN KEY (Building) REFERENCES Building(BuildingID)
- RoomType (Office, Lecture, Lab)

Lab (Room subtype)

- LabID PRIMARY KEY
- #WorkStations
- FOREIGN KEY (LABID) REFERENCES Room(RoomID)

Lecture (Room subtype)

- LecutureID PRIMARY KEY
- NumSeats
- FOREIGN KEY (LectureID) REFERENCES Room(RoomID)

Office (Room subtype)

- OfficeID PRIMARY KEY
- NumDesks
- FOREIGN KEY (OfficeID) REFERENCES Room(RoomID)

Major

- MajorID PRIMARY KEY,
- DeptID FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
- MajorName
- CreditsRequired

StudentMajor

- StudentID FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
- MajorID FOREIGN KEY (MajorID) REFERENCES Major(MajorID)
- DateOfDeclaration
- PRIMARY KEY (StudentID, MajorID)

Minor

- MinorID PRIMARY KEY,
- DeptID FOREIGN KEY (DeptID) REFERENCES Department(DeptID)
- MinorName
- CreditsRequired

StudentMinor

- StudentID FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
- MinorID FOREIGN KEY (MinorID) REFERENCES Minor(MinorID)
- DateOfDeclaration
- PRIMARY KEY (StudentID, MinorID)

MajorRequirement

- MajorID FOREIGN KEY (MajorID) REFERENCES Major(MajorID)
- CourseID FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
- MajorGrade
- PRIMARY KEY (MajorID, CourseID)

MinorRequirement

- MinorID FOREIGN KEY (MinorID) REFERENCES Minor(MinorID)
- CourseID FOREIGN KEY (CourseID) REFERENCES Course(CourseID)
- MinorGrade
- PRIMARY KEY (MinorID, CourseID)

DegreeAudit

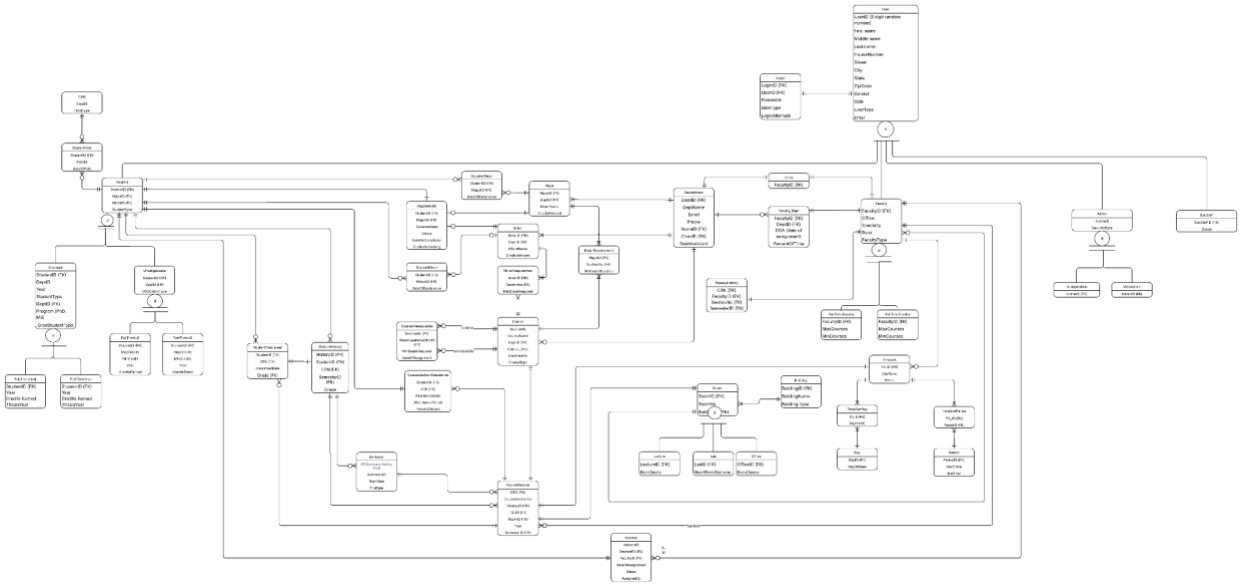
- DegreeAuditID PRIMARY KEY
- StudentID FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
- MajorID FOREIGN KEY (MajorID) REFERENCES Major(MajorID)
- MinorID FOREIGN KEY (MinorID) REFERENCES Minor(MinorID)
- GenerateDate
- Status
- Credits_Completed
- Credits_Remaining

Advisor

- AdvisorID
- StudentID
- DOA
- Status
- AssignedBy
- PRIMARY KEY (AdvisorID, StudentID)

- FOREIGN KEY (AdvisorID) REFERENCES Faculty(FacultyID)
- FOREIGN KEY (StudentID) REFERENCES Student(StudentID)
- FOREIGN KEY (AssignedBy) REFERENCES Admin(AdminID)

5. EERD



6. Schemas

Admin(AdminID PK, SecurityType, FK(AdminID) → Users(UserID))

Advisor(FacultyID PK, StudentID PK, DOA, Status, AssignedBy, FK(FacultyID) → Faculty(FacultyID), FK(StudentID) → Student(StudentID))

AuditLog(LogID PK, TableName, RecordID, Operation, ChangedBy, ChangeDate, Details)

Building(BuildingID PK, BuildingName, BuildingType)

Chair(ChairID PK, FacultyID, FK(FacultyID) → Faculty(FacultyID))

Course(CourseID PK, CourseName, DeptID, Course_Desc, Credits, CourseType, FK(DeptID) → Department(DeptID))

CoursePrerequisite(CourseID PK, PrerequisiteCourseID PK, MinGradeRequired, DecisionDate, FK(CourseID) → Course(CourseID), FK(PrerequisiteCourseID) → Course(CourseID))

CourseSection(CRN PK, CourseID, CourseSectionNo, FacultyID, TimeSlotID, RoomID, Year, SemesterID, AvailableSeats, Status,

FK(CourseID) → Course(CourseID), FK(FacultyID) → Faculty(FacultyID), FK(RoomID) → Room(RoomID), FK(SemesterID) → Semester(SemesterID))

CourseSectionAttendance(StudentID PK, CRN PK, CourseID, AttendanceDate, PresentAbsent,

FK(StudentID) → Student(StudentID), FK(CRN) → CourseSection(CRN))

coursesection_backup(CRN PK, CourseID, CourseSectionNo, FacultyID, TimeSlotID, RoomID, Year, SemesterID, AvailableSeats, Status)

Day(DayID PK, DayOfWeek UNIQUE)

DegreeAudit(DegreeAuditID PK, StudentID UNIQUE, MajorID, MinorID, GenerateDate, Status, Credits_Completed, Credits_Remaining, CumulativeGPA, SemesterGPA, Courses_Taken, Courses_Needed,

FK(StudentID) → Student(StudentID), FK(MajorID) → Major(MajorID), FK(MinorID) → Minor(MinorID))

Department(DeptID PK, DeptName, Email, Phone, RoomID, ChairID, DeptAssistant,

FK(RoomID) → Room(RoomID), FK(ChairID) → Chair(ChairID))

Faculty(FacultyID PK, OfficeID, Specialty, Ranking, FacultyType, FK(OfficeID) → Room(RoomID))

FacultyHistory(FacultyHistoryID PK, CRN, FacultyID, CourseID, SemesterID, Year, AssignedDate, Status, DeptID,

FK(CRN) → CourseSection(CRN), FK(FacultyID) → Faculty(FacultyID), FK(CourseID) → Course(CourseID), FK(SemesterID) → Semester(SemesterID), FK(DeptID) → Department(DeptID))

Faculty_Dept(FacultyID PK, DeptID PK, DOA, FK(FacultyID) → Faculty(FacultyID), FK(DeptID) → Department(DeptID))

FullTimeFaculty(FacultyID PK, MaxCourses, FK(FacultyID) → Faculty(FacultyID))

FullTimeGrad(StudentID PK, Year, CreditsEarned, ThesisYear, FK(StudentID) → Graduate(StudentID))

FullTimeUG(StudentID PK, MaxCredits, MinCredits, Year, CreditsEarned, FK(StudentID) → Undergraduate(StudentID))

GradingScale(GradeLetter PK, GradeValue)

Graduate(StudentID PK, DeptID, Year, Program, GradStudentType, FK(StudentID) → Student(StudentID), FK(DeptID) → Department(DeptID))

Hold(HoldID PK, HoldType)

Lab(LabID PK, NumWorkStations, FK(LabID) → Room(RoomID))

Lecture(LectureID PK, NumSeats, FK(LectureID) → Room(RoomID))

Login(LoginID PK, Email, UserID, UserType, Password, LoginAttempts, ResetToken, ResetExpiry, MustReset,

FK(LoginID) → Users(UserID), FK(UserID) → Users(UserID), FK(Email) → Users(Email))

Major(MajorID PK, DeptID, MajorName, CreditsNeeded, FK(DeptID) → Department(DeptID))

MajorRequirement(MajorID PK, CourseID PK, RequirementDescription, CreditsRequired, FK(MajorID) → Major(MajorID), FK(CourseID) → Course(CourseID))

Minor(MinorID PK, DeptID, MinorName, CreditsNeeded, FK(DeptID) → Department(DeptID))

MinorRequirement(MinorID PK, CourseID PK, RequirementDescription, CreditsRequired, FK(MinorID) → Minor(MinorID), FK(CourseID) → Course(CourseID))

Office(OfficeID PK, NumDesks, FK(OfficeID) → Room(RoomID))

PartTimeFaculty(FacultyID PK, MaxCourses, FK(FacultyID) → Faculty(FacultyID))

PartTimeGrad(StudentID PK, Year, CreditsEarned, ThesisYear, FK(StudentID) → Graduate(StudentID))

PartTimeUG(StudentID PK, MaxCredits, MinCredits, Year, CreditsEarned, FK(StudentID) → Undergraduate(StudentID))

Period(PeriodID PK, StartTime, EndTime)

Room(RoomID PK, RoomNo, BuildingID, RoomType, FK(BuildingID) → Building(BuildingID))

Semester(SemesterID PK, SemesterName, Year, StartDate, EndDate, AddDropDeadline)

StatStaff(StatStaffID PK, StaffName, Status)

Student(StudentID PK, MajorID, MinorID, StudentType, FK(MajorID) → Major(MajorID), FK(MinorID) → Minor(MinorID))

StudentEnrollment(StudentID PK, SemesterID PK, CRN PK, CourseID, Status, EnrollmentDate, Grade,

FK(StudentID) → Student(StudentID), FK(SemesterID) → Semester(SemesterID), FK(CRN) → CourseSection(CRN), FK(CourseID) → CourseSection(CourseID))

StudentHistory(HistoryID PK, StudentID, CRN, SemesterID, Grade, CourseID,

FK(StudentID) → StudentEnrollment(StudentID), FK(CRN) → StudentEnrollment(CRN), FK(SemesterID) → StudentEnrollment(SemesterID), FK(CourseID) → CourseSection(CourseID))

StudentHold(StudentID PK, HoldID PK, DateOfHold, FK(StudentID) → Student(StudentID), FK(HoldID) → Hold(HoldID))

StudentMajor(StudentID PK, MajorID PK, DateOfDeclaration, FK(MajorID) → Major(MajorID))

StudentMinor(StudentID PK, MinorID PK, DateOfDeclaration, FK(MinorID) → Minor(MinorID))

TimeSlot(TS_ID PK)

TimeSlotDay(TS_ID PK, DayID PK, FK(TS_ID) → TimeSlot(TS_ID), FK(DayID) → Day(DayID))

TimeSlotPeriod(TS_ID PK, PeriodID PK, FK(TS_ID) → TimeSlot(TS_ID), FK(PeriodID) → Period(PeriodID))

Undergraduate(StudentID PK, DeptID, UGStudentType, FK(StudentID) → Student(StudentID), FK(DeptID) → Department(DeptID))

UpdateAdmin(AdminID PK, FK(AdminID) → Admin(AdminID))

Users(UserID PK, FirstName, MiddleName, LastName, HouseNumber, Street, City, State, ZIP, Gender, DOB, UserType, Email UNIQUE, Status)

ViewAdmin(AdminID PK, FK(AdminID) → Admin(AdminID))

7. Create Table Queries

Table	Create table
Admin	CREATE TABLE `Admin` (`AdminID` int(10) unsigned NOT NULL, `SecurityType` enum('VIEW','UPDATE') NOT NULL, PRIMARY KEY (`AdminID`), CONSTRAINT `fk_admin_users` FOREIGN KEY (`AdminID`) REFERENCES `Users` (`UserID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Advisor	CREATE TABLE `Advisor` (`FacultyID` int(10) unsigned NOT NULL, `StudentID` int(10) unsigned NOT NULL, `DOA` date NOT NULL, `Status` enum('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE', `AssignedBy` int(11) NOT NULL, PRIMARY KEY (`FacultyID`,`StudentID`), KEY `StudentID` (`StudentID`), KEY `AssignedBy` (`AssignedBy`), CONSTRAINT `fk_facultyadvisor_users` FOREIGN KEY (`FacultyID`) REFERENCES `Faculty` (`FacultyID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_studentadvisor_users` FOREIGN KEY (`StudentID`) REFERENCES `Student` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

AuditLog	CREATE TABLE `AuditLog` (`LogID` int(11) NOT NULL AUTO_INCREMENT, `TableName` varchar(50) DEFAULT NULL, `RecordID` varchar(50) DEFAULT NULL, `Operation` enum('INSERT','UPDATE','DELETE') DEFAULT NULL, `ChangedBy` varchar(50) DEFAULT NULL, `ChangeDate` datetime DEFAULT current_timestamp(), `Details` text DEFAULT NULL, PRIMARY KEY (`LogID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Building	CREATE TABLE `Building` (`BuildingID` varchar(3) NOT NULL DEFAULT 'TBD', `BuildingName` varchar(50) NOT NULL, `BuildingType` enum('Academic') NOT NULL, PRIMARY KEY (`BuildingID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Chair	CREATE TABLE `Chair` (`ChairID` int(11) NOT NULL, `FacultyID` int(10) unsigned NOT NULL, PRIMARY KEY (`ChairID`), KEY `FacultyID` (`FacultyID`), CONSTRAINT `fk_facultychair_users` FOREIGN KEY (`FacultyID`) REFERENCES `Faculty` (`FacultyID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

Course	CREATE TABLE `Course` (`CourseID` varchar(10) NOT NULL, `CourseName` varchar(50) NOT NULL, `DeptID` int(11) NOT NULL, `Course_Desc` text DEFAULT NULL, `Credits` int(11) NOT NULL DEFAULT 3, `CourseType` enum('UNDERGRAD','GRAD') NOT NULL, PRIMARY KEY (`CourseID`), KEY `DeptID` (`DeptID`), CONSTRAINT `fk_course_deptid` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
CoursePrerequisite	CREATE TABLE `CoursePrerequisite` (`CourseID` varchar(10) NOT NULL, `PrerequisiteCourseID` varchar(10) NOT NULL, `MinGradeRequired` varchar(5) NOT NULL DEFAULT 'C', `DecisionDate` date DEFAULT NULL, PRIMARY KEY (`CourseID`,`PrerequisiteCourseID`), KEY `coursepreprerequisite_ibfk_2` (`PrerequisiteCourseID`), CONSTRAINT `fk_cp_courseID` FOREIGN KEY (`CourseID`) REFERENCES `Course` (`CourseID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_cp_prerequisitecourseID` FOREIGN KEY (`PrerequisiteCourseID`) REFERENCES `Course` (`CourseID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

CourseSection	CREATE TABLE `CourseSection` (`CRN` int(11) NOT NULL, `CourseID` varchar(10) NOT NULL, `CourseSectionNo` int(11) NOT NULL, `FacultyID` int(10) unsigned DEFAULT NULL, `TimeSlotID` int(11) DEFAULT NULL, `RoomID` varchar(16) DEFAULT NULL, `Year` int(11) DEFAULT NULL, `SemesterID` varchar(16) DEFAULT NULL, `AvailableSeats` int(11) DEFAULT NULL, `Status` enum('PLANNED','IN-PROGRESS','COMPLETED','CANCELLED') DEFAULT NULL, PRIMARY KEY (`CRN`), KEY `fk_course` (`CourseID`), KEY `fk_faculty` (`FacultyID`), KEY `fk_room` (`RoomID`), KEY `fk_semester` (`SemesterID`), CONSTRAINT `fk_course` FOREIGN KEY (`CourseID`) REFERENCES `Course` (`CourseID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_faculty` FOREIGN KEY (`FacultyID`) REFERENCES `Faculty` (`FacultyID`) ON DELETE SET NULL ON UPDATE CASCADE, CONSTRAINT `fk_room` FOREIGN KEY (`RoomID`) REFERENCES `Room` (`RoomID`) ON DELETE SET NULL ON UPDATE CASCADE, CONSTRAINT `fk_semester` FOREIGN KEY (`SemesterID`) REFERENCES `Semester` (`SemesterID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

CourseSectionAttendance	CREATE TABLE `CourseSectionAttendance` (`StudentID` int(10) unsigned NOT NULL, `CRN` int(11) NOT NULL, `CourseID` varchar(10) NOT NULL, `AttendanceDate` date NOT NULL, `PresentAbsent` enum('PRESENT','ABSENT') DEFAULT NULL, PRIMARY KEY (`StudentID`,`CRN`), KEY `CRN` (`CRN`), KEY `coursesectionattendance_ibfk_3` (`CourseID`), CONSTRAINT `fk_csa_crn` FOREIGN KEY (`CRN`) REFERENCES `CourseSection` (`CRN`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_csa_student` FOREIGN KEY (`StudentID`) REFERENCES `Student` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
coursesection_backup	CREATE TABLE `coursesection_backup` (`CRN` int(11) NOT NULL, `CourseID` varchar(10) NOT NULL, `CourseSectionNo` int(11) NOT NULL, `FacultyID` int(11) DEFAULT NULL, `TimeSlotID` varchar(16) DEFAULT NULL, `RoomID` varchar(16) DEFAULT NULL, `Year` int(11) DEFAULT NULL, `SemesterID` varchar(16) DEFAULT NULL, `AvailableSeats` int(11) DEFAULT NULL, `Status` enum('COMPLETED','IN-PROGRESS','CANCELLED','PLANNED') DEFAULT 'PLANNED',

	PRIMARY KEY (`CRN`), UNIQUE KEY `uq_cs_sem_crn` (`SemesterID`,`CRN`), UNIQUE KEY `uq_cs_sem_course_label` (`SemesterID`,`CourseID`,`CourseSectionNo`), KEY `FacultyID` (`FacultyID`), KEY `TimeSlotID` (`TimeSlotID`), KEY `RoomID` (`RoomID`), KEY `coursesection_ibfk_1` (`CourseID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Day	CREATE TABLE `Day` (`DayID` int(11) NOT NULL, `DayOfWeek` varchar(10) NOT NULL, PRIMARY KEY (`DayID`), UNIQUE KEY `DayOfWeek` (`DayOfWeek`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

DegreeAudit	CREATE TABLE `DegreeAudit` (`DegreeAuditID` int(11) NOT NULL AUTO_INCREMENT, `StudentID` int(10) unsigned NOT NULL, `MajorID` int(11) DEFAULT NULL, `MinorID` int(11) DEFAULT NULL, `GenerateDate` datetime NOT NULL DEFAULT current_timestamp() ON UPDATE current_timestamp(), `Status` varchar(32) NOT NULL DEFAULT 'ACTIVE', `Credits_Completed` int(11) NOT NULL DEFAULT 0, `Credits_Remaining` int(11) NOT NULL DEFAULT 0, `CumulativeGPA` int(11) DEFAULT 0, `SemesterGPA` int(11) DEFAULT 0, `Courses_Taken` text DEFAULT NULL, `Courses_Needed` text DEFAULT NULL, PRIMARY KEY (`DegreeAuditID`), UNIQUE KEY `uq_degreeaudit_student` (`StudentID`), KEY `fk_da_majorid` (`MajorID`), KEY `fk_da_minorid` (`MinorID`), CONSTRAINT `fk_da_majorid` FOREIGN KEY (`MajorID`) REFERENCES `Major` (`MajorID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_da_minorid` FOREIGN KEY (`MinorID`) REFERENCES `Minor` (`MinorID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_degreeaudit_users` FOREIGN KEY (`StudentID`) REFERENCES `Student` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB AUTO_INCREMENT=3201 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

Department	CREATE TABLE `Department` (`DeptID` int(11) NOT NULL, `DeptName` varchar(25) NOT NULL, `Email` varchar(255) NOT NULL, `Phone` varchar(20) NOT NULL, `RoomID` varchar(16) NOT NULL, `ChairID` int(11) NOT NULL, `DeptAssistant` varchar(15) NOT NULL, PRIMARY KEY (`DeptID`), KEY `RoomID` (`RoomID`), KEY `ChairID` (`ChairID`), CONSTRAINT `fk_dept_charid` FOREIGN KEY (`ChairID`) REFERENCES `Chair` (`ChairID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_dept_room` FOREIGN KEY (`RoomID`) REFERENCES `Room` (`RoomID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Faculty	CREATE TABLE `Faculty` (`FacultyID` int(10) unsigned NOT NULL, `OfficeID` varchar(16) NOT NULL, `Specialty` varchar(25) NOT NULL, `Ranking` varchar(25) NOT NULL, `FacultyType` enum('PartTimeFaculty','FullTimeFaculty') NOT NULL, PRIMARY KEY (`FacultyID`), KEY `idx_fac_office` (`OfficeID`), CONSTRAINT `fk_faculty_room` FOREIGN KEY (`OfficeID`) REFERENCES `Room` (`RoomID`) ON DELETE CASCADE ON UPDATE CASCADE

	) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
FacultyHistory	CREATE TABLE `FacultyHistory` (`CRN` int(11) NOT NULL, `FacultyID` int(10) unsigned DEFAULT NULL, `CourseID` varchar(10) DEFAULT NULL, `SemesterID` varchar(16) DEFAULT NULL, `FacultyHistoryID` int(11) NOT NULL AUTO_INCREMENT, `Year` int(11) DEFAULT NULL, `AssignedDate` datetime DEFAULT NULL, `Status` enum('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE', `DeptID` int(11) DEFAULT NULL, PRIMARY KEY (`FacultyHistoryID`), KEY `CRN` (`CRN`), KEY `SemesterID` (`SemesterID`), KEY `facultyhistory_ibfk_3` (`CourseID`), KEY `idx_fh_faculty` (`FacultyID`), KEY `idx_fh_dept` (`DeptID`), KEY `idx_fh_crn` (`CRN`), CONSTRAINT `fk_fh_crn` FOREIGN KEY (`CRN`) REFERENCES `CourseSection` (`CRN`) ON DELETE CASCADE ON UPDATE CASCADE,

	CONSTRAINT `fk_fh_dept` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`) ON DELETE SET NULL ON UPDATE CASCADE, CONSTRAINT `fk_fh_faculty` FOREIGN KEY (`FacultyID`) REFERENCES `Faculty` (`FacultyID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_fh_semesterid` FOREIGN KEY (`SemesterID`) REFERENCES `Semester` (`SemesterID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_fs_crn` FOREIGN KEY (`CRN`) REFERENCES `CourseSection` (`CRN`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Faculty_Dept	CREATE TABLE `Faculty_Dept` (`FacultyID` int(10) unsigned NOT NULL, `DeptID` int(11) NOT NULL, `DOA` date NOT NULL, PRIMARY KEY (`FacultyID`,`DeptID`), KEY `DeptID` (`DeptID`), CONSTRAINT `fk_faculty_deptid` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_facultydept_users` FOREIGN KEY (`FacultyID`) REFERENCES `Faculty` (`FacultyID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

FullTimeFaculty	CREATE TABLE `FullTimeFaculty` (`FacultyID` int(10) unsigned NOT NULL, `MaxCourses` int(11) NOT NULL, PRIMARY KEY (`FacultyID`), CONSTRAINT `fk_ftf_users` FOREIGN KEY (`FacultyID`) REFERENCES `Faculty` (`FacultyID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
FullTimeGrad	CREATE TABLE `FullTimeGrad` (`StudentID` int(10) unsigned NOT NULL, `Year` int(11) NOT NULL, `CreditsEarned` int(11) NOT NULL, `ThesisYear` int(11) DEFAULT NULL, PRIMARY KEY (`StudentID`), CONSTRAINT `fk_ftg_users` FOREIGN KEY (`StudentID`) REFERENCES `Graduate` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
FullTimeUG	CREATE TABLE `FullTimeUG` (`StudentID` int(10) unsigned NOT NULL, `MaxCredits` int(11) NOT NULL, `MinCredits` int(11) NOT NULL, `Year` enum('Freshman','Sophomore','Junior','Senior') NOT NULL, `CreditsEarned` int(11) NOT NULL, PRIMARY KEY (`StudentID`), CONSTRAINT `fk_ftu_users` FOREIGN KEY (`StudentID`) REFERENCES `Undergraduate` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

GradingScale	CREATE TABLE `GradingScale` (`GradeLetter` varchar(2) NOT NULL, `GradeValue` decimal(3,2) DEFAULT NULL, PRIMARY KEY (`GradeLetter`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Graduate	CREATE TABLE `Graduate` (`StudentID` int(10) unsigned NOT NULL, `DeptID` int(11) NOT NULL, `Year` int(11) NOT NULL, `Program` enum('PhD','MS') NOT NULL, `GradStudentType` enum('FullTimeGrad','PartTimeGrad') NOT NULL, PRIMARY KEY (`StudentID`), KEY `DeptID` (`DeptID`), CONSTRAINT `fk_grad_deptid` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_graduate_users` FOREIGN KEY (`StudentID`) REFERENCES `Student` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Hold	CREATE TABLE `Hold` (`HoldID` int(11) NOT NULL, `HoldType` enum('FINANCIAL','ACADEMIC','HEALTH') NOT NULL, PRIMARY KEY (`HoldID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

Lab	CREATE TABLE `Lab` (`LabID` varchar(16) NOT NULL, `NumWorkStations` int(11) NOT NULL, PRIMARY KEY (`LabID`), CONSTRAINT `fk_Lab_room` FOREIGN KEY (`LabID`) REFERENCES `Room` (`RoomID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Lecture	CREATE TABLE `Lecture` (`LectureID` varchar(16) NOT NULL, `NumSeats` int(11) NOT NULL, PRIMARY KEY (`LectureID`), CONSTRAINT `fk_lecture_room` FOREIGN KEY (`LectureID`) REFERENCES `Room` (`RoomID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Login	CREATE TABLE `Login` (`LoginID` int(10) unsigned NOT NULL, `Email` varchar(100) NOT NULL, `UserID` int(10) unsigned NOT NULL, `UserType` varchar(50) NOT NULL, `Password` varchar(250) NOT NULL, `LoginAttempts` int(11) NOT NULL DEFAULT 0, `ResetToken` char(64) DEFAULT NULL, `ResetExpiry` datetime DEFAULT NULL, `MustReset` tinyint(1) NOT NULL DEFAULT 0, PRIMARY KEY (`LoginID`),

	KEY `UserID` (`UserID`), KEY `fk_login_email` (`Email`), CONSTRAINT `fk_login_email` FOREIGN KEY (`Email`) REFERENCES `Users` (`Email`), CONSTRAINT `fk_login_loginID` FOREIGN KEY (`LoginID`) REFERENCES `Users` (`UserID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_login_userID` FOREIGN KEY (`UserID`) REFERENCES `Users` (`UserID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Major	CREATE TABLE `Major` (`MajorID` int(11) NOT NULL, `DeptID` int(11) NOT NULL, `MajorName` varchar(50) NOT NULL, `CreditsNeeded` int(11) NOT NULL, PRIMARY KEY (`MajorID`), KEY `DeptID` (`DeptID`), CONSTRAINT `fk_major_deptid` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
MajorRequirement	CREATE TABLE `MajorRequirement` (`MajorID` int(11) NOT NULL, `CourseID` varchar(10) NOT NULL, `RequirementDescription` text NOT NULL, `CreditsRequired` int(11) NOT NULL, PRIMARY KEY (`MajorID`,`CourseID`),

	KEY `majorrequirement_ibfk_2` (`CourseID`), CONSTRAINT `fk_majorrequirement` FOREIGN KEY (`MajorID`) REFERENCES `Major` (`MajorID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_mr_courseid` FOREIGN KEY (`CourseID`) REFERENCES `Course` (`CourseID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Minor	CREATE TABLE `Minor` (`MinorID` int(11) NOT NULL, `DeptID` int(11) NOT NULL, `MinorName` varchar(50) NOT NULL, `CreditsNeeded` int(11) NOT NULL, PRIMARY KEY (`MinorID`), KEY `DeptID` (`DeptID`), CONSTRAINT `fk_minor_deptid` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
MinorRequirement	CREATE TABLE `MinorRequirement` (`MinorID` int(11) NOT NULL, `CourseID` varchar(10) NOT NULL, `RequirementDescription` text NOT NULL, `CreditsRequired` int(11) DEFAULT 3, PRIMARY KEY (`MinorID`,`CourseID`), KEY `minorrequirement_ibfk_2` (`CourseID`), CONSTRAINT `fk_minorrequirement` FOREIGN KEY (`MinorID`) REFERENCES `Minor` (`MinorID`) ON DELETE CASCADE ON UPDATE CASCADE,

	CONSTRAINT `fk_mnr_courseid` FOREIGN KEY (`CourseID`) REFERENCES `Course` (`CourseID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Office	CREATE TABLE `Office` (`OfficeID` varchar(16) NOT NULL, `NumDesks` int(11) NOT NULL, PRIMARY KEY (`OfficeID`), CONSTRAINT `fk_office_roomid` FOREIGN KEY (`OfficeID`) REFERENCES `Room` (`RoomID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
PartTimeFaculty	CREATE TABLE `PartTimeFaculty` (`FacultyID` int(10) unsigned NOT NULL, `MaxCourses` int(11) NOT NULL, PRIMARY KEY (`FacultyID`), CONSTRAINT `fk_ptf_users` FOREIGN KEY (`FacultyID`) REFERENCES `Faculty` (`FacultyID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
PartTimeGrad	CREATE TABLE `PartTimeGrad` (`StudentID` int(10) unsigned NOT NULL, `Year` int(11) NOT NULL, `CreditsEarned` int(11) NOT NULL, `ThesisYear` int(11) NOT NULL, PRIMARY KEY (`StudentID`),

	CONSTRAINT `fk_ptg_users` FOREIGN KEY (`StudentID`) REFERENCES `Graduate` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
PartTimeUG	CREATE TABLE `PartTimeUG` (`StudentID` int(10) unsigned NOT NULL, `MaxCredits` int(11) NOT NULL, `MinCredits` int(11) NOT NULL, `Year` enum('Freshman','Sophomore','Junior','Senior') NOT NULL, `CreditsEarned` int(11) NOT NULL, PRIMARY KEY (`StudentID`), CONSTRAINT `fk_ptu_users` FOREIGN KEY (`StudentID`) REFERENCES `Undergraduate` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Period	CREATE TABLE `Period` (`PeriodID` int(11) NOT NULL, `StartTime` time NOT NULL, `EndTime` time NOT NULL, PRIMARY KEY (`PeriodID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

Room	CREATE TABLE `Room` (`RoomID` varchar(16) NOT NULL, `RoomNo` varchar(16) NOT NULL DEFAULT 'TBD', `BuildingID` varchar(3) NOT NULL DEFAULT 'TBD', `RoomType` enum('Lecture','Office','Lab') NOT NULL, PRIMARY KEY (`RoomID`), UNIQUE KEY `uq_building_room` (`BuildingID`,`RoomNo`), KEY `BuildingID` (`BuildingID`), KEY `idx_room_roomtype` (`RoomType`), CONSTRAINT `fk_room_buildingid` FOREIGN KEY (`BuildingID`) REFERENCES `Building` (`BuildingID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Semester	CREATE TABLE `Semester` (`SemesterID` varchar(16) NOT NULL, `SemesterName` enum('SPRING','FALL') NOT NULL, `Year` int(11) NOT NULL, `StartDate` date DEFAULT NULL, `EndDate` date DEFAULT NULL, `AddDropDeadline` datetime DEFAULT NULL, PRIMARY KEY (`SemesterID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
StatStaff	CREATE TABLE `StatStaff` (`StatStaffID` int(10) unsigned NOT NULL, `StaffName` varchar(25) NOT NULL, `Status` enum('Active','Inactive') NOT NULL,

	PRIMARY KEY (`StatStaffID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Student	CREATE TABLE `Student` (`StudentID` int(10) unsigned NOT NULL, `MajorID` int(11) NOT NULL, `MinorID` int(11) DEFAULT NULL, `StudentType` enum('Graduate','Undergraduate') NOT NULL, PRIMARY KEY (`StudentID`), KEY `MajorID` (`MajorID`), KEY `MinorID` (`MinorID`), CONSTRAINT `fk_student_majorid` FOREIGN KEY (`MajorID`) REFERENCES `Major` (`MajorID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_student_minorid` FOREIGN KEY (`MinorID`) REFERENCES `Minor` (`MinorID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
StudentEnrollment	CREATE TABLE `StudentEnrollment` (`StudentID` int(10) unsigned NOT NULL, `SemesterID` varchar(16) NOT NULL, `CRN` int(11) NOT NULL, `CourseID` varchar(16) NOT NULL, `Status` enum('ENROLLED','DROPPED','WAITLIST','COMPLETED') DEFAULT NULL, `EnrollmentDate` date NOT NULL, `Grade` varchar(2) DEFAULT NULL, PRIMARY KEY (`StudentID`,`SemesterID`,`CRN`),

StudentHistory	KEY `SemesterID` (`CRN`), KEY `ix_se_sem_crn` (`SemesterID`,`CRN`), KEY `fk_se_courseID` (`CourseID`), CONSTRAINT `fk_se_courseID` FOREIGN KEY (`CourseID`) REFERENCES `CourseSection` (`CourseID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_se_semesterID` FOREIGN KEY (`SemesterID`) REFERENCES `Semester` (`SemesterID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_sse_crn` FOREIGN KEY (`CRN`) REFERENCES `CourseSection` (`CRN`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_studentenrollment_users` FOREIGN KEY (`StudentID`) REFERENCES `Student` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci CREATE TABLE `StudentHistory` (`HistoryID` int(11) NOT NULL AUTO_INCREMENT, `StudentID` int(10) unsigned NOT NULL, `CRN` int(11) NOT NULL, `SemesterID` varchar(16) DEFAULT NULL, `Grade` varchar(2) DEFAULT NULL, `CourseID` varchar(16) NOT NULL, PRIMARY KEY (`HistoryID`), UNIQUE KEY `uq_studenthistory` (`StudentID`,`CRN`,`SemesterID`), KEY `StudentID` (`StudentID`), KEY `CRN` (`CRN`), KEY `ix_sh_sem_crn` (`SemesterID`,`CRN`), KEY `fk_sh_courseID` (`CourseID`),

	CONSTRAINT `fk_sh_courseID` FOREIGN KEY (`CourseID`) REFERENCES `CourseSection` (`CourseID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_sh_crn` FOREIGN KEY (`CRN`) REFERENCES `StudentEnrollment` (`CRN`) ON UPDATE CASCADE, CONSTRAINT `fk_sh_semeseterid` FOREIGN KEY (`SemesterID`) REFERENCES `StudentEnrollment` (`SemesterID`) ON UPDATE CASCADE, CONSTRAINT `fk_sh_studentID` FOREIGN KEY (`StudentID`) REFERENCES `StudentEnrollment` (`StudentID`) ON UPDATE CASCADE) ENGINE=InnoDB AUTO_INCREMENT=241988 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
StudentHold	CREATE TABLE `StudentHold` (`StudentID` int(10) unsigned NOT NULL, `HoldID` int(11) NOT NULL, `DateOfHold` date NOT NULL, PRIMARY KEY (`StudentID`,`HoldID`), KEY `HoldID` (`HoldID`), CONSTRAINT `fk_holdid` FOREIGN KEY (`HoldID`) REFERENCES `Hold` (`HoldID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_studenthold_users` FOREIGN KEY (`StudentID`) REFERENCES `Student` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
StudentMajor	CREATE TABLE `StudentMajor` (`StudentID` int(10) unsigned NOT NULL, `MajorID` int(11) NOT NULL, `DateOfDeclaration` date NOT NULL,

	PRIMARY KEY (`StudentID`,`MajorID`), KEY `MajorID` (`MajorID`), CONSTRAINT `fk_studentmajor` FOREIGN KEY (`MajorID`) REFERENCES `Major` (`MajorID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
StudentMinor	CREATE TABLE `StudentMinor` (`StudentID` int(10) unsigned NOT NULL, `MinorID` int(11) NOT NULL, `DateOfDeclaration` date NOT NULL, PRIMARY KEY (`StudentID`,`MinorID`), KEY `MinorID` (`MinorID`), CONSTRAINT `fk_studentminor` FOREIGN KEY (`MinorID`) REFERENCES `Minor` (`MinorID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
TimeSlot	CREATE TABLE `TimeSlot` (`TS_ID` int(11) NOT NULL, PRIMARY KEY (`TS_ID`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
TimeSlotDay	CREATE TABLE `TimeSlotDay` (`TS_ID` int(11) NOT NULL, `DayID` int(11) NOT NULL, PRIMARY KEY (`TS_ID`,`DayID`), KEY `DayID` (`DayID`), CONSTRAINT `fk_tsd_day` FOREIGN KEY (`DayID`) REFERENCES `Day` (`DayID`) ON DELETE CASCADE ON UPDATE CASCADE,

	CONSTRAINT `fk_tsd_timelsot` FOREIGN KEY (`TS_ID`) REFERENCES `TimeSlot` (`TS_ID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
TimeSlotPeriod	CREATE TABLE `TimeSlotPeriod` (`TS_ID` int(11) NOT NULL, `PeriodID` int(11) NOT NULL, PRIMARY KEY (`TS_ID`,`PeriodID`), KEY `PeriodID` (`PeriodID`), CONSTRAINT `fk_tsp` FOREIGN KEY (`PeriodID`) REFERENCES `Period` (`PeriodID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_tsp_timelsot` FOREIGN KEY (`TS_ID`) REFERENCES `TimeSlot` (`TS_ID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Undergraduate	CREATE TABLE `Undergraduate` (`StudentID` int(10) unsigned NOT NULL, `DeptID` int(11) NOT NULL, `UGStudentType` enum('FullTimeUG','PartTimeUG') NOT NULL, PRIMARY KEY (`StudentID`), KEY `DeptID` (`DeptID`), CONSTRAINT `fk_ug_deptid` FOREIGN KEY (`DeptID`) REFERENCES `Department` (`DeptID`) ON DELETE CASCADE ON UPDATE CASCADE, CONSTRAINT `fk_undergraduate_users` FOREIGN KEY (`StudentID`) REFERENCES `Student` (`StudentID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci

UpdateAdmin	CREATE TABLE `UpdateAdmin` (`AdminID` int(10) unsigned NOT NULL, PRIMARY KEY (`AdminID`), CONSTRAINT `fk_updateadmin_users` FOREIGN KEY (`AdminID`) REFERENCES `Admin` (`AdminID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
Users	CREATE TABLE `Users` (`UserID` int(10) unsigned NOT NULL AUTO_INCREMENT, `FirstName` varchar(50) NOT NULL, `MiddleName` varchar(50) NOT NULL, `LastName` varchar(50) NOT NULL, `HouseNumber` int(11) NOT NULL, `Street` varchar(255) NOT NULL, `City` varchar(255) NOT NULL, `State` varchar(255) NOT NULL, `ZIP` int(11) NOT NULL, `Gender` enum('M','F') NOT NULL, `DOB` date NOT NULL, `UserType` enum('Student','Faculty','Admin','StatStaff') NOT NULL, `Email` varchar(100) NOT NULL, `Status` enum('ACTIVE','INACTIVE') NOT NULL DEFAULT 'ACTIVE', PRIMARY KEY (`UserID`), UNIQUE KEY `Email` (`Email`), UNIQUE KEY `uq_users_email` (`Email`)

	) ENGINE=InnoDB AUTO_INCREMENT=999697 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci
ViewAdmin	CREATE TABLE `ViewAdmin` (`AdminID` int(10) unsigned NOT NULL, PRIMARY KEY (`AdminID`), CONSTRAINT `fk_viewadmin_users` FOREIGN KEY (`AdminID`) REFERENCES `Admin` (`AdminID`) ON DELETE CASCADE ON UPDATE CASCADE) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_general_ci